LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 9: HMVC, AUTHORIZATION, DAN AJAX PADA LARAVEL REACT STARTER KIT



Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan

Laporan ini berisi pengerjaan Modul 9 bagian 9.5 Membangun Aplikasi HMVC React sampai 9.6 Pengujian Otomatis dengan Pest. Project yang saya pakai adalah Laravel React Starter Kit di folder pertemuan9_hmvc_react_2, dengan database MySQL hmvc_react. Tujuannya sebagai berikut:

1. Memasang nwidart/laravel-modules pada React Starter Kit dan mengatur Inertia agar halaman React di dalam folder Modules dapat dibuka dengan format NamaModul::halaman.
2. Menambahkan kolom role pada tabel users, menyesuaikan CreateNewUser milik Fortify, dan membagikan data pengguna serta pesan flash ke seluruh halaman React lewat HandleInertiaRequests.
3. Membuat middleware role beserta aliasnya, lalu memakainya untuk membedakan halaman admin dan halaman yang boleh dibuka semua pengguna yang sudah login.
4. Membangun modul UserManagement dengan CRUD berbasis Inertia useForm(), termasuk aturan agar admin tidak dapat menurunkan role atau menghapus akunnya sendiri.
5. Membangun modul Pendaftaran dengan CRUD AJAX memakai fetch(), unggah berkas, serta pesan validasi 422 yang tampil di bawah input tanpa menutup dialog.
6. Menulis pengujian otomatis dengan Pest untuk role, middleware, CRUD Inertia, dan CRUD AJAX, lalu menjalankan seluruh pengujian sampai lulus.

2. Alat dan Bahan

Praktikum dikerjakan pada satu laptop Lenovo 83DV dengan prosesor Intel Core i7-13650HX, RAM 12 GB, dan Windows 11 Pro versi 10.0.26100. Perangkat lunak yang dipakai adalah Laragon sebagai server lokal dengan MySQL 8.4.3 dan phpMyAdmin 5.2.3; PHP 8.3.30 (ZTS); Composer 2.10.3; Node.js 24.22.1 dan npm 11.19.1; Laravel Framework 13.40.0; inertiajs/inertia-laravel 3.7.0; laravel/fortify 1.41.0; nwidart/laravel-modules 13.3.0; React 19.2.0; @inertiajs/react 3.0.0; Tailwind CSS 4; TypeScript 5.7.2; Pest 4.7.8; Visual Studio Code; Google Chrome; serta Git.

Aplikasi memakai database MySQL hmvc_react dengan APP_LOCALE=id. Server pengembangan dijalankan dengan php artisan serve pada alamat http://127.0.0.1:8000, sedangkan aset React dikompilasi oleh Vite. Berkas uji unggahan berupa rapor_budi.png (7,4 KB), surat_keterangan.pdf (454 byte), dan berkas.zip (124 byte) untuk menguji penolakan format.

3. Langkah-Langkah Praktikum

Pengerjaan dimulai dari project React Starter Kit yang sudah dibuat, lalu dilanjutkan dengan pemasangan package modul, penyesuaian authentication, middleware role, dua modul, dan pengujian. Seluruh perintah dijalankan dari terminal di dalam folder project.

3.1 Menyiapkan Modul dan Halaman React di Dalamnya

Package modul dipasang dengan composer config allow-plugins.wikimedia/composer-merge-plugin true dan composer require nwidart/laravel-modules, lalu konfigurasinya dipublikasikan memakai php artisan vendor:publish --provider="Nwidart\Modules\LaravelModulesServiceProvider". Bagian merge-plugin pada composer.json berisi include Modules/*/composer.json, dan composer dump-autoload dijalankan setiap kali file itu berubah.

Dua modul dibuat dengan php artisan module:make UserManagement Pendaftaran. Folder resources/views, resources/assets, vite.config.js, dan package.json milik masing-masing modul saya hapus karena tampilan modul ditulis dengan React, lalu folder resources/js/pages dibuat di dalam kedua modul. Isi routes/api.php dikosongkan karena modul ini tidak menyediakan endpoint API.

Agar Inertia dapat menemukan halaman di dalam modul, bagian resolve pada resources/js/app.tsx diubah. import.meta.glob mengumpulkan halaman dari resources/js/pages dan dari Modules/*/resources/js/pages, sedangkan nama yang mengandung :: dipecah menjadi nama modul dan nama halaman.

```
13 void createInertiaApp({
14     title: (title) => (title ? `${title} - ${appName}` : appName),
15     resolve: (name) => {
16         const pages = import.meta.glob([
17             "./pages/**/*.tsx",
18             "../../Modules/*/resources/js/pages/**/*.tsx",
19         ]) as Record<string, () => Promise<ComponentType<any>>>;
20 
21         // Halaman modul ditulis dengan format NamaModul::nama-halaman
22         if (name.includes("::")) {
23             const [module, page] = name.split("::");
24 
25             return resolvePageComponent<ComponentType<any>>(
26                 `../../Modules/${module}/resources/js/pages/${page}.tsx`,
27                 pages,
28             );
29         }
30 
31         return resolvePageComponent<ComponentType<any>>(
32             `./pages/${name}.tsx`,
33             pages,
34         );
35     },
```

Class Tailwind yang ditulis di halaman modul hanya ikut dibuat kalau foldernya dipindai. Karena itu baris @source '../../Modules/*/resources/js/**/*.{ts,tsx}' ditambahkan pada resources/css/app.css, dan folder modul didaftarkan pada bagian include tsconfig.json agar TypeScript mengenali file .tsx di dalamnya.

Satu pengaturan tambahan saya lakukan pada resources/views/app.blade.php. File bawaan starter kit menambahkan halaman yang sedang dibuka sebagai entri Vite tersendiri, sedangkan halaman modul tidak punya entri di manifest karena dimuat lewat import.meta.glob. Tanpa perubahan ini, halaman modul berhenti dengan error 500 Unable to locate file in Vite manifest. Entri per halaman sekarang hanya ditambahkan untuk halaman yang tidak mengandung ::.

```
37         @fonts
38 
39         @viteReactRefresh
40         @php
41             // Halaman modul (NamaModul::halaman) dimuat lewat import.meta.glob pada
                   app.tsx,
42             // jadi tidak punya entri tersendiri di manifest Vite.
43             $pageEntry = str_contains($page['component'], '::')
44                 ? []
45                 : ["resources/js/pages/{$page['component']}.tsx"];
46         @endphp
47         @vite(['resources/css/app.css', 'resources/js/app.tsx', ...$pageEntry])
```

3.2 Menyesuaikan Authentication Fortify dan Role Pengguna

Register, login, dan logout tetap memakai Fortify beserta halaman React bawaannya. Yang saya tambahkan adalah kolom role pada tabel users dengan nilai awal user dan posisi setelah email.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user')->after('email');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

Daftar role yang sah disimpan pada enum UserRole. Method label() mengembalikan tulisan Admin atau User yang dipakai pada badge dan pilihan role.

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}
```

Model User memakai cast role ke enum tersebut, ditambah nilai awal role pada atribut dan dua method pemeriksaan. isAdmin() dipakai controller dan Form Request, sedangkan hasRole() dipakai middleware. Properti fillable dan hidden ditulis dengan atribut PHP karena starter kit ini memakai gaya Laravel 13.

```
 */
#[Fillable(['name', 'email', 'role', 'password'])]
#[Hidden(['password', 'two_factor_secret', 'two_factor_recovery_codes',
        'remember_token'])]
class User extends Authenticatable implements PasskeyUser
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable, PasskeyAuthenticatable, TwoFactorAuthenticatable;

    /**
     * The model's default attribute values.
     *
     * @var array<string, mixed>
     */
    protected $attributes = [
        'role' => 'user',
    ];
```

```
    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'two_factor_confirmed_at' => 'datetime',
            'role' => UserRole::class,
        ];
    }

    /**
     * Determine whether the user has the admin role.
     */
    public function isAdmin(): bool
    {
        return $this->role === UserRole::Admin;
    }

```

```php
    /**
     * Determine whether the user has one of the given roles.
     */
    public function hasRole(UserRole|string ...$roles): bool
    {
        foreach ($roles as $role) {
            if ($this->role === ($role instanceof UserRole ? $role :
                    UserRole::tryFrom($role))) {
                return true;
            }
        }

        return false;
    }
}

```

CreateNewUser diubah supaya akun hasil register selalu berperan user. Field role tidak dibaca dari input, jadi pengguna tidak dapat mendaftarkan diri sebagai admin walaupun menambahkan role=admin pada form.

```php
     * @param  array<string, string>  $input
     */
    public function create(array $input): User
    {
        Validator::make($input, [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email'],
            'password' => ['required', 'string', 'confirmed', Password::defaults()],
        ])->validate();

        return User::create([
            'name' => $input['name'],
            'email' => $input['email'],
            'password' => $input['password'],
            'role' => UserRole::User,
        ]);
    }
}
```

Pesan error login bawaan Fortify belum diterjemahkan. File lang/id/auth.php dibuat agar pesan gagal login dan batas percobaan tampil dalam Bahasa Indonesia sesuai APP_LOCALE=id.

```php
<?php

return [
    'failed' => 'Email atau password salah.',
    'password' => 'Password yang dimasukkan salah.',
    'throttle' => 'Terlalu banyak percobaan login. Coba lagi dalam :seconds detik.',
];
```

Data pengguna yang sedang login dan pesan flash dibagikan lewat method share() pada HandleInertiaRequests. Key lain bawaan starter kit dibiarkan seperti semula.

```
    {
        return [
            ...parent::share($request),
            'name' => config('app.name'),
            'auth' => [
                'user' => $request->user(),
            ],
            'flash' => [
                'success' => fn () => $request->session()->get('success'),
                'error' => fn () => $request->session()->get('error'),
            ],
            'sidebarOpen' => ! $request->hasCookie('sidebar_state') ||
                    $request->cookie('sidebar_state') === 'true',
        ];
    }
}
```

Hook useSharedProps() membaca shared props tersebut dan mengembalikan user, isAdmin, dan flash. Nilai isAdmin hanya dipakai untuk menyembunyikan menu, sedangkan penjagaan sebenarnya tetap dilakukan middleware di sisi server.

```
import { usePage } from '@inertiajs/react';

export type UserRole = 'admin' | 'user';

export type AuthUser = {
    id: number;
    name: string;
    email: string;
    role: UserRole;
};

type SharedProps = {
    auth: { user: AuthUser };
    flash: { success?: string | null; error?: string | null };
};

export function useSharedProps() {
    const { auth, flash } = usePage<SharedProps>().props;

    return {
        user: auth.user,
        isAdmin: auth.user?.role === 'admin',
        flash,
    };
}
```

Menu sidebar disusun dari array mainNavItems. Menu Kelola User hanya ditambahkan kalau isAdmin bernilai true, dan tautan Repository serta Documentation bawaan starter kit dihapus.

```
export function AppSidebar() {
    const { isAdmin } = useSharedProps();

    const mainNavItems: NavItem[] = [
        { title: 'Dashboard', href: '/dashboard', icon: LayoutGrid },
        { title: 'Pendaftaran', href: '/pendaftaran', icon: ClipboardList },
        ...(isAdmin
            ? [{ title: 'Kelola User', href: '/admin/users', icon: Users }]
            : []),
    ];

    return (
        <Sidebar collapsible="icon" variant="inset">
```

Dashboard memakai controller invokable yang mengirim jumlah pendaftar dan jumlah user. Jumlah user hanya dihitung untuk admin, sedangkan untuk user nilainya null sehingga kartunya tidak dirender.

```
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Models\Daftar;

class DashboardController extends Controller
{
    /**
     * Show the dashboard with record counts per module.
     */
    public function __invoke(Request $request): Response
    {
        return Inertia::render('dashboard', [
            'totalPendaftar' => Daftar::count(),
            'totalUsers' => $request->user()->isAdmin() ? User::count() : null,
        ]);
    }
}
```

Route dashboard pada routes/web.php diarahkan ke DashboardController. Route login, register, dan logout tidak ditulis di file ini karena sudah didaftarkan Fortify, dan route modul juga tidak ditulis di sini karena setiap modul memuat route-nya sendiri.

3.3 Membuat Middleware Role

Middleware EnsureUserHasRole menerima daftar role sebagai parameter. Jika pengguna belum login atau role-nya tidak termasuk yang diizinkan, request dihentikan dengan abort(403) beserta pesan berbahasa Indonesia.

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     *
     * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();

        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}
```

Middleware didaftarkan dengan alias role di bootstrap/app.php. Baris bawaan starter kit, termasuk HandleInertiaRequests pada middleware web, dibiarkan karena shared props auth dan flash bergantung pada baris itu.

```php
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->encryptCookies(except: ['appearance', 'sidebar_state']);

        $middleware->web(append: [
            HandleAppearance::class,
            HandleInertiaRequests::class,
            AddLinkHeadersForPreloadedAssets::class,
        ]);

        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        $exceptions->shouldRenderJsonWhen(
            fn (Request $request) => $request->is('api/*') || $request->expectsJson(),
```

Urutan middleware pada route selalu auth lebih dulu, baru role. Dengan urutan itu tamu diarahkan ke halaman login, sedangkan pengguna yang sudah login tetapi role-nya tidak sesuai mendapat halaman 403.

3.4 Membuat Modul UserManagement (CRUD Inertia)

Dua Form Request dibuat di dalam modul. StoreUserRequest mewajibkan nama, email unik, role yang hanya boleh admin atau user, dan password dengan konfirmasi. Method authorize() pada keduanya memeriksa isAdmin() sebagai lapisan pengaman kedua di belakang middleware.

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email'],
            'role' => ['required', Rule::enum(UserRole::class)],
            'password' => ['required', 'confirmed', Password::defaults()],
        ];
    }
```

```php
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}

```

UpdateUserRequest memakai aturan unique yang mengabaikan email milik user yang sedang diedit, dan password boleh kosong. Ada satu aturan tambahan: kalau admin mengubah akunnya sendiri, role hanya boleh tetap admin. Pesan untuk aturan itu ditulis lewat method messages().

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * An admin cannot demote their own account, so they never lock themselves out.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $user = $this->route('user');
        $isEditingOwnAccount = $user?->is($this->user()) ?? false;

        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', Rule::unique('users',
                    'email')->ignore($user?->id)],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->value])),
            ],
            'password' => ['nullable', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'role.in' => 'Anda tidak dapat menurunkan role akun Anda sendiri.',
        ];
    }

```

```php
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}

```

UserManagementController menyediakan daftar, tambah, edit, update, dan hapus. Method index() menyaring data berdasarkan kata kunci pada nama atau email dan filter role, mengurutkan hasil berdasarkan nama, lalu mengirim sepuluh baris per halaman dengan withQueryString() agar filter tetap terbawa saat berpindah halaman. through() memilih kolom yang dikirim ke React supaya password dan remember_token tidak ikut.

```php
    {
        $search = $request->string('search')->trim()->toString();
        $role = UserRole::tryFrom($request->string('role')->toString());

        $users = User::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('name', 'like', "%{$search}%")
                        ->orWhere('email', 'like', "%{$search}%");
                });
            })
            ->when($role, fn (Builder $query) => $query->where('role', $role))
            ->orderBy('name')
            ->paginate(10)
            ->withQueryString()
            ->through(fn (User $user) => [
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email,
                'role' => $user->role->value,
                'role_label' => $user->role->label(),
                'created_at' => $user->created_at?->diffForHumans(),
            ]);
```

```php
        return Inertia::render('UserManagement::index', [
            'users' => $users,
            'filters' => [
                'search' => $search,
                'role' => $role?->value ?? '',
            ],
            'roles' => $this->roleOptions(),
        ]);
    }

    /**
     * Show the form for creating a new user.
     */
    public function create(): Response
    {
        return Inertia::render('UserManagement::create', [
            'roles' => $this->roleOptions(),
        ]);
    }

    /**
     * Store a newly created user.
     */
    public function store(StoreUserRequest $request): RedirectResponse
    {
        User::create($request->validated());

        return redirect()->route('admin.users.index')->with('success', 'User berhasil
                ditambahkan.');
```

```php
    }

    /**
     * Show the form for editing the specified user.
     */
    public function edit(User $user): Response
    {
        return Inertia::render('UserManagement::edit', [
            'user' => [
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email,
                'role' => $user->role->value,
            ],
            'roles' => $this->roleOptions(),
        ]);
    }

    /**
     * Update the specified user. The password is only changed when filled in.
     */
    public function update(UpdateUserRequest $request, User $user): RedirectResponse
    {
        $attributes = $request->validated();

        if (blank($attributes['password'] ?? null)) {
            unset($attributes['password']);
        }

        $user->update($attributes);
```

```
104         return redirect()->route('admin.users.index')->with('success', 'User berhasil
105                 diperbarui.');
        }
106
107     /**
108      * Remove the specified user. An admin cannot delete their own account.
109      */
110     public function destroy(Request $request, User $user): RedirectResponse
111     {
112         if ($user->is($request->user())) {
113             return redirect()->route('admin.users.index')->with('error', 'Anda tidak
114                     dapat menghapus akun Anda sendiri.');
            }
115
116         $user->delete();
117
118         return redirect()->route('admin.users.index')->with('success', 'User berhasil
119                 dihapus.');
        }
120
```

```
121     /**
122      * Role options for the select input on the React page.
123      *
124      * @return list<array{value: string, label: string}>
125      */
126     private function roleOptions(): array
127     {
128         return array_map(
129             fn (UserRole $role) => ['value' => $role->value, 'label' => $role->label()],
130             UserRole::cases(),
131         );
132     }
133 }
134
```

destroy() menolak penghapusan akun sendiri dengan pesan error, sedangkan roleOptions() menyiapkan pasangan value dan label untuk elemen select pada form React.
Route modul memakai middleware auth dan role:admin, prefix admin, serta nama route berawalan admin.. Route::resource dipakai tanpa method show karena halaman detail tidak diperlukan.

```
1 <?php
2
3 use Illuminate\Support\Facades\Route;
4 use Modules\UserManagement\Http\Controllers\UserManagementController;
5
6 Route::middleware(['auth', 'role:admin'])
7     ->prefix('admin')
8     ->name('admin.')
9     ->group(function () {
10         Route::resource('users', UserManagementController::class)->except('show');
11     });
```

Komponen UserForm dipakai bersama oleh halaman tambah dan edit. Komponen ini hanya menampilkan input, sedangkan data dan proses pengiriman diatur oleh halaman yang memakainya. Setiap input punya InputError yang menampilkan pesan dari objek errors milik useForm(), dan atribut noValidate mematikan validasi browser agar seluruh pesan berasal dari Form Request.

```tsx
import { Link } from '@inertiajs/react';
import type { FormEvent } from 'react';
import InputError from '@/components/input-error';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';

export type UserFormData = {
    name: string;
    email: string;
    role: string;
    password: string;
    password_confirmation: string;
};

type UserFormProps = {
    data: UserFormData;
    errors: Partial<Record<keyof UserFormData, string>>;
    processing: boolean;
    roles: { value: string; label: string }[];
    isEdit?: boolean;
    submitLabel: string;
    setData: (key: keyof UserFormData, value: string) => void;
    onSubmit: (event: FormEvent) => void;
};
```

```tsx
export function UserForm({
    data,
    errors,
    processing,
    roles,
    isEdit = false,
    submitLabel,
    setData,
    onSubmit,
}: UserFormProps) {
    return (
        <form
            onSubmit={onSubmit}
            noValidate
            className="max-w-xl space-y-5 rounded-xl border p-6"
        >
            <div className="grid gap-2">
                <Label htmlFor="name">
                    Nama <span className="text-red-500">*</span>
                </Label>
                <Input
                    id="name"
                    value={data.name}
                    onChange={(event) => setData('name', event.target.value)}
                    autoFocus
                />
                <InputError message={errors.name} />
            </div>

            <div className="grid gap-2">
                <Label htmlFor="email">
                    Email <span className="text-red-500">*</span>
                </Label>
                <Input
                    id="email"
                    type="email"
                    value={data.email}
                    onChange={(event) => setData('email', event.target.value)}
                />
                <InputError message={errors.email} />
            </div>

            <div className="grid gap-2">
                <Label htmlFor="role">
                    Role <span className="text-red-500">*</span>
                </Label>
                <select
                    id="role"
                    value={data.role}
                    onChange={(event) => setData('role', event.target.value)}
                    className="h-9 rounded-md border border-input bg-transparent px-3
                            text-sm shadow-xs"
                >
                    {roles.map((role) => (
```

```
                        <option key={role.value} value={role.value}>
                            {role.label}
                        </option>
                    ))}
                </select>
                <InputError message={errors.role} />
            </div>

            <div className="grid gap-2">
                <Label htmlFor="password">
                    Password {!isEdit && <span className="text-red-500">*</span>}
                </Label>
                <Input
                    id="password"
                    type="password"
                    value={data.password}
                    onChange={(event) => setData('password', event.target.value)}
                />
                {isEdit && (
                    <p className="text-xs text-muted-foreground">
                        Kosongkan jika tidak ingin mengganti password.
                    </p>
                )}
                <InputError message={errors.password} />
            </div>
```

```
            <div className="grid gap-2">
                <Label htmlFor="password_confirmation">Konfirmasi Password</Label>
                <Input
                    id="password_confirmation"
                    type="password"
                    value={data.password_confirmation}
                    onChange={(event) =>
                        setData('password_confirmation', event.target.value)
                    }
                />
            </div>

            <div className="flex gap-2">
                <Button type="submit" disabled={processing}>
                    {processing ? 'Menyimpan...' : submitLabel}
                </Button>
                <Button asChild variant="secondary">
                    <Link href="/admin/users">Batal</Link>
                </Button>
            </div>
        </form>
    );
}

```

Halaman create mengisi form kosong dengan role bawaan user, lalu mengirim isinya dengan post('/admin/users'). Token CSRF ditangani Inertia, jadi tidak ada tag csrf seperti pada Blade. Halaman edit memakai put() dan mengisi form dari data yang dikirim method edit(), dengan password dibiarkan kosong.

```tsx
import { Head, useForm } from '@inertiajs/react';
import type { FormEvent } from 'react';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';
import { UserForm } from '../components/user-form';
import type { UserFormData } from '../components/user-form';

type CreateProps = {
    roles: { value: string; label: string }[];
};

const breadcrumbs: BreadcrumbItem[] = [
    { title: 'Kelola User', href: '/admin/users' },
    { title: 'Tambah User', href: '/admin/users/create' },
];

export default function Create({ roles }: CreateProps) {
    const { data, setData, post, processing, errors } = useForm<UserFormData>({
        name: '',
        email: '',
        role: 'user',
        password: '',
        password_confirmation: '',
    });

    const submit = (event: FormEvent) => {
        event.preventDefault();
        post('/admin/users');
    };
```

```tsx
    return (
        <AppLayout breadcrumbs={breadcrumbs}>
            <Head title="Tambah User" />

            <div className="flex flex-1 flex-col gap-4 p-4">
                <div>
                    <h1 className="text-xl font-semibold">Tambah User</h1>
                    <p className="text-sm text-muted-foreground">
                        Isi data akun baru beserta role-nya.
                    </p>
                </div>

                <UserForm
                    data={data}
                    errors={errors}
                    processing={processing}
                    roles={roles}
                    submitLabel="Simpan"
                    setData={setData}
                    onSubmit={submit}
                />
            </div>
        </AppLayout>
    );
}
```

Halaman index memakai router.get() untuk pencarian dan filter role, serta router.delete() untuk hapus. preserveState menjaga isi kotak pencarian, sedangkan onChange pada select role membuat daftar langsung diperbarui. Dialog konfirmasi hapus baru mengirim request setelah tombol Ya, hapus ditekan.

```
38 export default function Index({ users, filters, roles }: IndexProps) {
39     const { user: currentUser } = useSharedProps();
40     const [search, setSearch] = useState(filters.search);
41     const [deleting, setDeleting] = useState<UserRow | null>(null);
42     const [processing, setProcessing] = useState(false);
43 
44     // Kirim ulang request GET /admin/users dengan kata kunci dan filter role
45     const applyFilters = (params: { search?: string; role?: string }) => {
46         router.get(
47             '/admin/users',
48             { ...filters, search, ...params },
49             { preserveState: true, replace: true },
50         );
51     };
52 
53     const handleSearch = (event: FormEvent) => {
54         event.preventDefault();
55         applyFilters({ search });
56     };
57 
58     const handleDelete = () => {
59         if (!deleting) {
60             return;
61         }
62 
```

```
63         router.delete(`/admin/users/${deleting.id}`, {
64             preserveScroll: true,
65             onStart: () => setProcessing(true),
66             onFinish: () => {
67                 setProcessing(false);
68                 setDeleting(null);
69             },
70         });
71     };
72 
73     return (
74         <AppLayout breadcrumbs={breadcrumbs}>
75             <Head title="Kelola User" />
76 
77             <div className="flex flex-1 flex-col gap-4 p-4">
78                 <div className="flex flex-wrap items-center justify-between gap-2">
79                     <div>
80                         <h1 className="text-xl font-semibold">Kelola User</h1>
81                         <p className="text-sm text-muted-foreground">
82                             Total {users.total} user terdaftar.
83                         </p>
84                     </div>
85                     <Button asChild>
86                         <Link href="/admin/users/create">
87                             <Plus /> Tambah User
88                         </Link>
89                     </Button>
90 
```

3.5 Membuat Modul Pendaftaran (CRUD AJAX dan Unggah Berkas)

Tabel daftars menyimpan nama, asal_sekolah, prodi_tujuan, email unik, dan no_hp sepanjang 20 karakter. Migration kedua menambahkan kolom berkas untuk path hasil store() dan nama_berkas untuk nama asli file, keduanya nullable.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('daftars', function (Blueprint $table) {
            $table->id();
            $table->string('nama');
            $table->string('asal_sekolah');
            $table->string('prodi_tujuan');
            $table->string('email')->unique();
            $table->string('no_hp', 20);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('daftars');
```

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('daftars', function (Blueprint $table) {
            $table->string('berkas')->nullable()->after('no_hp');
            $table->string('nama_berkas')->nullable()->after('berkas');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('daftars', function (Blueprint $table) {
            $table->dropColumn(['berkas', 'nama_berkas']);
        });
    }
};
```

Model Daftar mengaitkan factory milik modul lewat newFactory(), dan menambahkan atribut berkas_url pada output JSON. Nilainya dibentuk dari asset('storage/'.$this->berkas) sehingga React dapat langsung membuat tautan tanpa menyusun URL sendiri.

```php
<?php

namespace Modules\Pendaftaran\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Pendaftaran\Database\Factories\DaftarFactory;

class Daftar extends Model
{
    /** @use HasFactory<DaftarFactory> */
    use HasFactory;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'nama',
        'asal_sekolah',
        'prodi_tujuan',
        'email',
        'no_hp',
        'berkas',
        'nama_berkas',
    ];
```

```
    /**
     * The accessors to append to the model's array and JSON form.
     *
     * @var list<string>
     */
    protected $appends = [
        'berkas_url',
    ];

    /**
     * Public URL of the uploaded file, or null when there is no file.
     */
    protected function berkasUrl(): Attribute
    {
        return Attribute::get(fn (): ?string => $this->berkas ?
                asset('storage/'.$this->berkas) : null);
    }

    protected static function newFactory(): DaftarFactory
    {
        return DaftarFactory::new();
    }
}

```

DaftarRequest dipakai untuk tambah dan edit. Berkas wajib saat menambah dan boleh kosong saat edit, dengan format jpg, jpeg, png, atau pdf dan ukuran maksimal 2 MB. Nomor HP dibatasi regex 8 sampai 20 karakter berisi angka atau tanda plus, dan pesan untuk aturan berkas serta no hp ditulis sendiri.

```
<?php

namespace Modules\Pendaftaran\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class DaftarRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * The file is required when creating, and optional when updating (keep the old
            file).
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $daftar = $this->route('daftar');
```

```
        return [
            'nama' => ['required', 'string', 'max:255'],
            'asal_sekolah' => ['required', 'string', 'max:255'],
            'prodi_tujuan' => ['required', 'string', 'max:255'],
            'email' => ['required', 'email', 'max:255', Rule::unique('daftars',
                    'email')->ignore($daftar?->id)],
            'no_hp' => ['required', 'string', 'regex:/^[0-9+]{8,20}$/'],
            'berkas' => [$daftar ? 'nullable' : 'required', 'file',
                    'mimes:jpg,jpeg,png,pdf', 'max:2048'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'no_hp.regex' => 'No HP hanya boleh berisi angka (8-20 digit).',
            'berkas.required' => 'Berkas wajib diunggah.',
            'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png, atau pdf.',
            'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }
}
```

PendaftaranController mengembalikan JSON, bukan view atau redirect. index() hanya merender halaman React sekali tanpa data, getData() mengirim seluruh baris terbaru, dan edit() mengirim satu baris untuk mengisi dialog. store() dan update() menyimpan berkas ke disk public di folder berkas_pendaftaran, menghapus berkas lama ketika diganti, sedangkan destroy() menghapus baris beserta berkasnya.

```php
<?php

namespace Modules\Pendaftaran\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Http\Requests\DaftarRequest;
use Modules\Pendaftaran\Models\Daftar;

class PendaftaranController extends Controller
{
    /**
     * Display the React page. Data is loaded via AJAX.
     */
    public function index(): Response
    {
        return Inertia::render('Pendaftaran::index');
    }

    /**
     * Return all registrations as JSON.
     */
    public function getData(): JsonResponse
    {
        return response()->json(Daftar::latest()->get());
    }
```

```php
    /**
     * Store a newly created registration together with its uploaded file.
     */
    public function store(DaftarRequest $request): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        $file = $request->file('berkas');
        $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
        $data['nama_berkas'] = $file->getClientOriginalName();

        Daftar::create($data);

        return response()->json(['success' => 'Data berhasil disimpan.']);
    }

    /**
     * Return a single registration for the edit form.
     */
    public function edit(Daftar $daftar): JsonResponse
    {
        return response()->json($daftar);
    }

```

```php
    /**
     * Update the specified registration. The old file is replaced only when a new one
            is uploaded.
     */
    public function update(DaftarRequest $request, Daftar $daftar): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        if ($request->hasFile('berkas')) {
            if ($daftar->berkas) {
                Storage::disk('public')->delete($daftar->berkas);
            }

            $file = $request->file('berkas');
            $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
            $data['nama_berkas'] = $file->getClientOriginalName();
        }

        $daftar->update($data);

        return response()->json(['success' => 'Data berhasil diperbarui.']);
    }

```

```php
    /**
     * Remove the specified registration and its uploaded file.
     */
    public function destroy(Daftar $daftar): JsonResponse
    {
        if ($daftar->berkas) {
            Storage::disk('public')->delete($daftar->berkas);
        }

        $daftar->delete();

        return response()->json(['success' => 'Data berhasil dihapus.']);
    }
}

```

Route pendaftaran dapat diakses admin maupun user yang sudah login. Route pendaftaran/data diletakkan sebelum route yang memakai parameter {daftar} agar kata data tidak dianggap sebagai id.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Pendaftaran\Http\Controllers\PendaftaranController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('pendaftaran', [PendaftaranController::class,
            'index'])->name('pendaftaran.index');
    Route::get('pendaftaran/data', [PendaftaranController::class,
            'getData'])->name('pendaftaran.data');
    Route::post('pendaftaran', [PendaftaranController::class,
            'store'])->name('pendaftaran.store');
    Route::get('pendaftaran/{daftar}/edit', [PendaftaranController::class,
            'edit'])->name('pendaftaran.edit');
    Route::put('pendaftaran/{daftar}', [PendaftaranController::class,
            'update'])->name('pendaftaran.update');
    Route::delete('pendaftaran/{daftar}', [PendaftaranController::class,
            'destroy'])->name('pendaftaran.destroy');
});
```

Helper http.ts membungkus fetch(). Token CSRF dibaca dari cookie XSRF-TOKEN dan dikirim sebagai header X-XSRF-TOKEN, header Accept berisi application/json agar error validasi kembali sebagai JSON 422, dan status respons diperiksa manual karena fetch() tidak menganggap kode 4xx sebagai kegagalan.

```ts
export type ValidationErrors = Record<string, string[]>;

type ErrorBody = { message?: string; errors?: ValidationErrors } | null;

export class HttpError extends Error {
    status: number;
    data: ErrorBody;

    constructor(status: number, data: ErrorBody) {
        super(data?.message ?? `Request gagal dengan status ${status}`);
        this.status = status;
        this.data = data;
    }
}

// Membaca token CSRF dari cookie XSRF-TOKEN yang dibuat oleh Laravel
function getXsrfToken(): string {
    const cookie = document.cookie
        .split('; ')
        .find((row) => row.startsWith('XSRF-TOKEN='));

    return cookie ? decodeURIComponent(cookie.substring('XSRF-TOKEN='.length)) : '';
}

type RequestOptions = {
    method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
    body?: FormData;
};
```

```ts
export async function http<T>(
    url: string,
    { method = 'GET', body }: RequestOptions = {},
): Promise<T> {
    const response = await fetch(url, {
        method,
        body,
        credentials: 'same-origin',
        headers: {
            Accept: 'application/json',
            'X-Requested-With': 'XMLHttpRequest',
            'X-XSRF-TOKEN': getXsrfToken(),
        },
    });

    const data = await response.json().catch(() => null);

    if (!response.ok) {
        throw new HttpError(response.status, data);
    }

    return data as T;
}

```

Halaman Pendaftaran menyimpan data server, kata kunci, isi form, berkas, pesan, dan status dialog pada state. loadData() dipanggil sekali lewat useEffect, lalu dipanggil lagi setiap kali data berhasil disimpan atau dihapus. Pencarian memakai useMemo sehingga penyaringan berjalan di browser tanpa request baru.

```ts
type Daftar = {
    id: number;
    nama: string;
    asal_sekolah: string;
    prodi_tujuan: string;
    email: string;
    no_hp: string;
    berkas: string | null;
    nama_berkas: string | null;
    berkas_url: string | null;
};

type FormFields = Pick<Daftar, 'nama' | 'asal_sekolah' | 'prodi_tujuan' | 'email' |
        'no_hp'>;

type Message = { type: 'success' | 'error'; text: string } | null;

const emptyForm: FormFields = {
    nama: '',
    asal_sekolah: '',
    prodi_tujuan: '',
    email: '',
    no_hp: '',
};
```

```
const fields: { name: keyof FormFields; label: string; type?: string }[] = [
    { name: 'nama', label: 'Nama' },
    { name: 'asal_sekolah', label: 'Asal Sekolah' },
    { name: 'prodi_tujuan', label: 'Prodi Tujuan' },
    { name: 'email', label: 'Email', type: 'email' },
    { name: 'no_hp', label: 'No HP' },
];

const urls = {
    data: '/pendaftaran/data',
    store: '/pendaftaran',
    edit: (id: number) => `/pendaftaran/${id}/edit`,
    update: (id: number) => `/pendaftaran/${id}`,
    destroy: (id: number) => `/pendaftaran/${id}`,
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Pendaftaran', href: '/pendaftaran' }];

// Tautan berkas yang dibuka di tab baru, ikon dipilih berdasarkan ekstensi
function FileLink({ url, name }: { url: string | null; name: string | null }) {
    if (!url) {
        return <span className="text-muted-foreground">-</span>;
    }

    const Icon = name?.toLowerCase().endsWith('.pdf') ? FileText : FileImage;

    return (
        <a
            href={url}
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex items-center gap-1 text-primary underline-offset-4
                    hover:underline"
        >
            <Icon className="size-4" /> {name ?? 'Lihat berkas'}
        </a>
    );
}

export default function Index() {
    const [records, setRecords] = useState<Daftar[]>([]);
    const [loading, setLoading] = useState(true);
    const [keyword, setKeyword] = useState('');
    const [message, setMessage] = useState<Message>(null);

    const [dialogOpen, setDialogOpen] = useState(false);
    const [editing, setEditing] = useState<Daftar | null>(null);
    const [form, setForm] = useState<FormFields>(emptyForm);
    const [file, setFile] = useState<File | null>(null);
    const [fileInputKey, setFileInputKey] = useState(0);
    const [errors, setErrors] = useState<ValidationErrors>({});
    const [saving, setSaving] = useState(false);

    const [deleting, setDeleting] = useState<Daftar | null>(null);
    const [processingDelete, setProcessingDelete] = useState(false);

```

```
    const showError = (error: unknown, fallback: string) => {
        const text =
            error instanceof HttpError ? (error.data?.message ?? fallback) : fallback;
        setMessage({ type: 'error', text });
    };

```

Form dikirim dengan FormData karena isinya memuat berkas. Saat mengedit, _method=PUT ditambahkan pada FormData dan request tetap dikirim sebagai POST karena PHP hanya membaca file multipart pada request POST. Error 422 disimpan ke state errors sehingga pesan muncul di bawah input tanpa menutup dialog.

```
    // Menyimpan data (tambah dan edit) menggunakan FormData
    const handleSubmit = async (event: FormEvent) => {
        event.preventDefault();
        setSaving(true);
        setErrors({});

        const formData = new FormData();
        Object.entries(form).forEach(([key, value]) => formData.append(key, value));

        if (file) {
            formData.append('berkas', file);
        }

        if (editing) {
            formData.append('_method', 'PUT');
        }

        try {
            const result = await http<{ success: string }>(
                editing ? urls.update(editing.id) : urls.store,
                {
                    method: 'POST',
                    body: formData,
                },
            );

            setDialogOpen(false);
            setMessage({ type: 'success', text: result.success });
            await loadData();
```

```
208         } catch (error) {
209             if (error instanceof HttpError && error.status === 422) {
210                 setErrors(error.data?.errors ?? {});
211             } else {
212                 showError(error, 'Terjadi kesalahan, silakan coba lagi.');
213             }
214         } finally {
215             setSaving(false);
216         }
217     };
218
219     // Menghapus data (DELETE /pendaftaran/{id}) setelah dikonfirmasi
220     const handleDelete = async () => {
221         if (!deleting) {
222             return;
223         }
224
225         setProcessingDelete(true);
226
227         try {
228             const result = await http<{ success: string }>(urls.destroy(deleting.id), {
229                 method: 'DELETE',
230             });
231
232             setMessage({ type: 'success', text: result.success });
233             await loadData();
234         } catch (error) {
235             showError(error, 'Data gagal dihapus.');
```

```
236         } finally {
237             setProcessingDelete(false);
238             setDeleting(null);
239         }
240     };
```

3.6 Menjalankan Aplikasi

Database dikosongkan dan diisi ulang dengan php artisan migrate:fresh --seed. Perintah itu menjalankan seluruh migration, termasuk dua migration milik modul Pendaftaran, lalu membuat akun admin@example.com dan user@example.com dengan password password. php artisan module:list menunjukkan kedua modul berstatus Enabled.

Aplikasi dijalankan dengan php artisan serve pada port 8000 dan npm run dev untuk Vite. Selama Vite berjalan, perubahan file .tsx langsung terlihat di browser tanpa refresh. Setelah itu saya menguji alur authentication, authorization, CRUD Inertia, dan CRUD AJAX seperti yang terdokumentasi pada bagian 4.

3.7 Menulis Pengujian Otomatis dengan Pest

Pengujian memakai RefreshDatabase yang sudah diaktifkan pada tests/Pest.php, sehingga setiap pengujian berjalan pada database kosong. File pertama menguji role dan modul UserManagement: tamu yang membuka halaman modul diarahkan ke login, akun hasil register selalu berperan user walaupun mengirim role=admin, user biasa mendapat 403, admin dapat membuka daftar dan menambah user, serta admin tidak dapat menurunkan role atau menghapus akunnya sendiri.

```php
<?php

use App\Models\User;
use Inertia\Testing\AssertableInertia as Assert;

test('guest is redirected to login from module pages', function () {
    $this->get('/pendaftaran')->assertRedirect(route('login'));
    $this->get('/admin/users')->assertRedirect(route('login'));
});

test('registered user always gets the user role', function () {
    $this->post('/register', [
        'name' => 'Mahasiswa',
        'email' => 'mahasiswa@example.com',
        'password' => 'password',
        'password_confirmation' => 'password',
        'role' => 'admin',
    ]);

    expect(User::where('email',
            'mahasiswa@example.com')->first()->isAdmin())->toBeFalse();
});

test('user cannot open the user management page', function () {
    $this->actingAs(User::factory()->create())
        ->get('/admin/users')
        ->assertForbidden();
});
```

```php
test('admin can see the user list rendered by the module page', function () {
    $this->actingAs(User::factory()->admin()->create())
        ->get('/admin/users')
        ->assertOk()->assertInertia(fn(Assert $page) => $page
            ->component('UserManagement::index', false)
            ->has('users.data', 1));
});

test('admin can create a user', function () {
    $this->actingAs(User::factory()->admin()->create())
        ->post('/admin/users', [
            'name' => 'User Baru',
            'email' => 'baru@example.com',
            'role' => 'user',
            'password' => 'password',
            'password_confirmation' => 'password',
        ])
        ->assertRedirect('/admin/users')
        ->assertSessionHas('success');

    $this->assertDatabaseHas('users', ['email' => 'baru@example.com', 'role' =>
    'user']);
});

test('admin cannot demote or delete their own account', function () {
    $admin = User::factory()->admin()->create();

    $this->actingAs($admin)
        ->put("/admin/users/{$admin->id}", [
            'name' => $admin->name,
            'email' => $admin->email,
            'role' => 'user',
        ])
        ->assertSessionHasErrors('role');

    $this->actingAs($admin)
        ->delete("/admin/users/{$admin->id}")
        ->assertSessionHas('error');

    $this->assertDatabaseHas('users', ['id' => $admin->id, 'role' => 'admin']);
});
```

File kedua menguji modul Pendaftaran. Storage::fake('public') dipakai agar berkas hasil unggahan tidak mengisi folder storage asli. Pengujiannya mencakup render halaman modul, endpoint JSON, validasi 422 saat form kosong, penyimpanan data beserta berkas, penolakan file zip, penggantian berkas lama saat update, penghapusan data beserta berkasnya, dan id yang tidak dikenal yang menghasilkan 404.

```php
<?php

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Filesystem\FilesystemAdapter;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;
use Inertia\Testing\AssertableInertia as Assert;
use Modules\Pendaftaran\Models\Daftar;

uses(RefreshDatabase::class);

beforeEach(function () {
    Storage::fake('public');
    $this->actingAs(User::factory()->create());
});

/**
 * Storage::disk() bertipe kontrak Filesystem, sedangkan assertExists()
 * dan assertMissing() hanya ada pada FilesystemAdapter.
 */
function fakePublicDisk(): FilesystemAdapter
{
    /** @var FilesystemAdapter $disk */
    $disk = Storage::disk('public');

    return $disk;
}
```

```php
function dataPendaftar(array $overrides = []): array
{
    return [
        'nama' => 'Budi Santoso',
        'asal_sekolah' => 'SMA Negeri 1',
        'prodi_tujuan' => 'Informatika',
        'email' => 'budi@example.com',
        'no_hp' => '081234567890',
        ...$overrides,
    ];
}

test('index renders the react page of the module', function () {
    $this->get('/pendaftaran')
        ->assertOk()
        ->assertInertia(fn (Assert $page) => $page->component(
            'Pendaftaran::index',
            false
        ));
});

test('data endpoint returns registrations as json', function () {
    Daftar::factory()->count(3)->create();

    $this->getJson('/pendaftaran/data')->assertOk()->assertJsonCount(3);
});

```

```php
test('store returns 422 when the form is empty', function () {
    $this->postJson('/pendaftaran', [])
        ->assertStatus(422)
        ->assertJsonValidationErrors([
            'nama',
            'asal_sekolah',
            'prodi_tujuan',
            'email',
            'no_hp',
            'berkas',
        ]);
});

test('store saves the data and the uploaded file', function () {
    $this->post('/pendaftaran', dataPendaftar([
        'berkas' => UploadedFile::fake()->image('rapor_budi.png'),
    ]), ['Accept' => 'application/json'])
        ->assertOk()
        ->assertJson(['success' => 'Data berhasil disimpan.']);

    $daftar = Daftar::firstOrFail();

    expect($daftar->nama_berkas)->toBe('rapor_budi.png');
    fakePublicDisk()->assertExists($daftar->berkas);
});

```

```php
test('zip file is rejected', function () {
    $this->post('/pendaftaran', dataPendaftar([
        'berkas' => UploadedFile::fake()->create('berkas.zip', 100),
    ]), ['Accept' => 'application/json'])
        ->assertStatus(422)
        ->assertJsonValidationErrors('berkas');

    $this->assertDatabaseCount('daftars', 0);
});

test('update replaces the old file', function () {
    $this->post('/pendaftaran', dataPendaftar([
        'berkas' => UploadedFile::fake()->image('lama.png'),
    ]), ['Accept' => 'application/json'])->assertOk();

    $daftar = Daftar::firstOrFail();
    $oldPath = $daftar->berkas;

    $this->post("/pendaftaran/{$daftar->id}", dataPendaftar([
        '_method' => 'PUT',
        'prodi_tujuan' => 'Sistem Informasi',
        'berkas' => UploadedFile::fake()->create(
            'surat_keterangan.pdf',

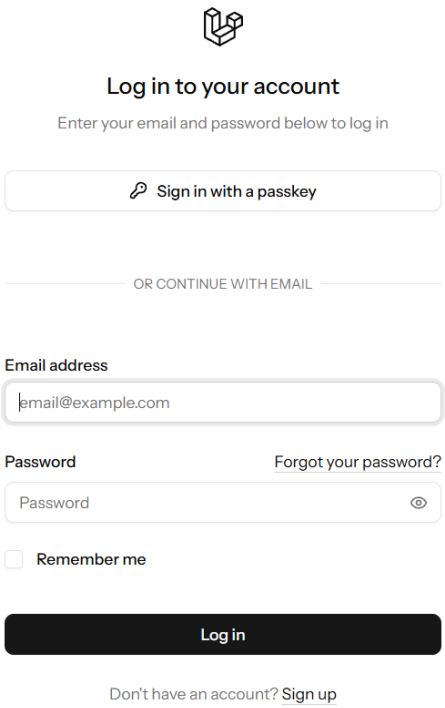
```

Seluruh pengujian dijalankan dengan php artisan test. Hasilnya 53 pengujian lulus dengan 199 assertion, mencakup pengujian authentication dan settings bawaan starter kit.

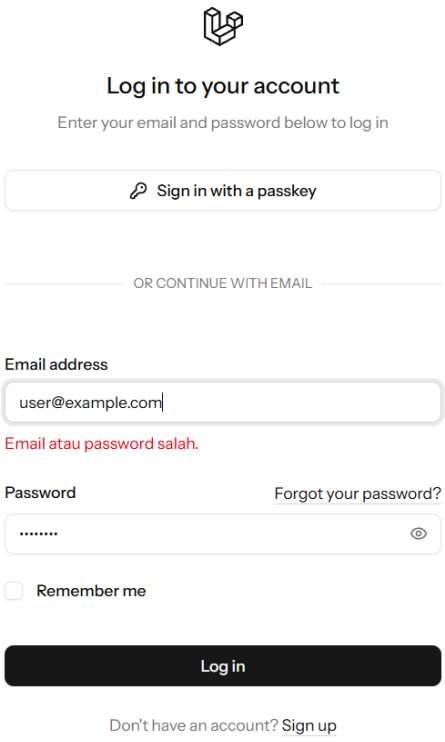
4. Hasil dan Dokumentasi Program

Seluruh halaman diuji pada browser dengan dua akun, yaitu Administrator sebagai admin dan Anggota Perpustakaan sebagai user. Akun user kedua dibuat melalui halaman register sehingga role-nya otomatis user.

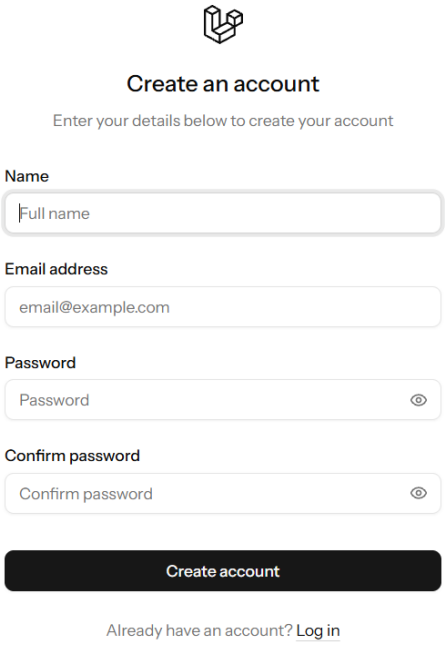
Halaman login dan register masih memakai tampilan bawaan starter kit. Pesan error login sudah berbahasa Indonesia setelah lang/id/auth.php ditambahkan, sedangkan pesan validasi register bawaan Fortify masih tampil dalam bahasa Inggris.



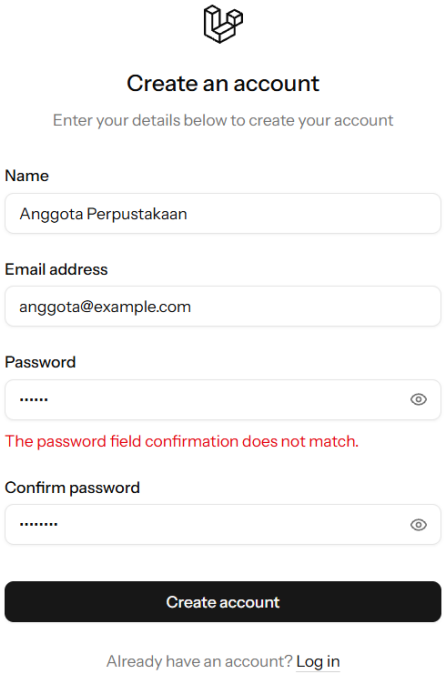
Gambar 1. Halaman login bawaan Fortify



Gambar 2. Pesan error ketika password yang dimasukkan salah

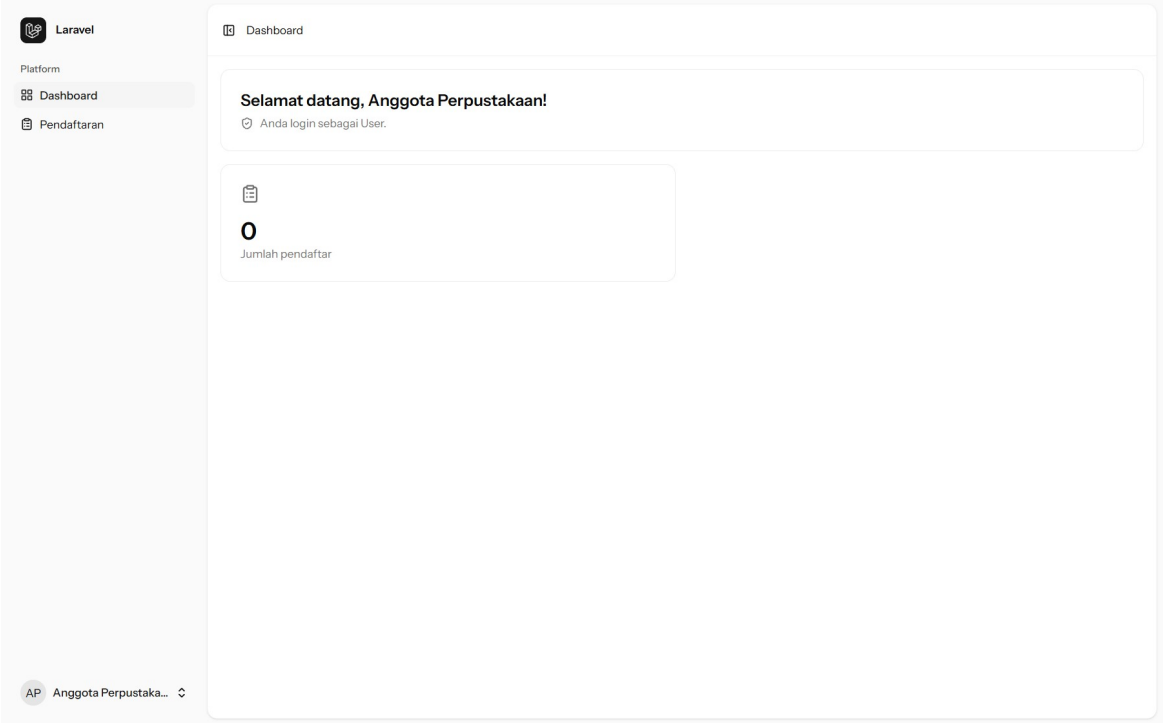


Gambar 3. Halaman register



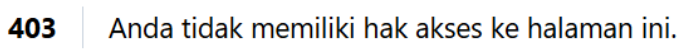
Gambar 4. Pesan validasi ketika konfirmasi password berbeda

Setelah register, pengguna langsung masuk ke dashboard sebagai User. Kartu jumlah user dan menu Kelola User tidak muncul karena isAdmin bernilai false.



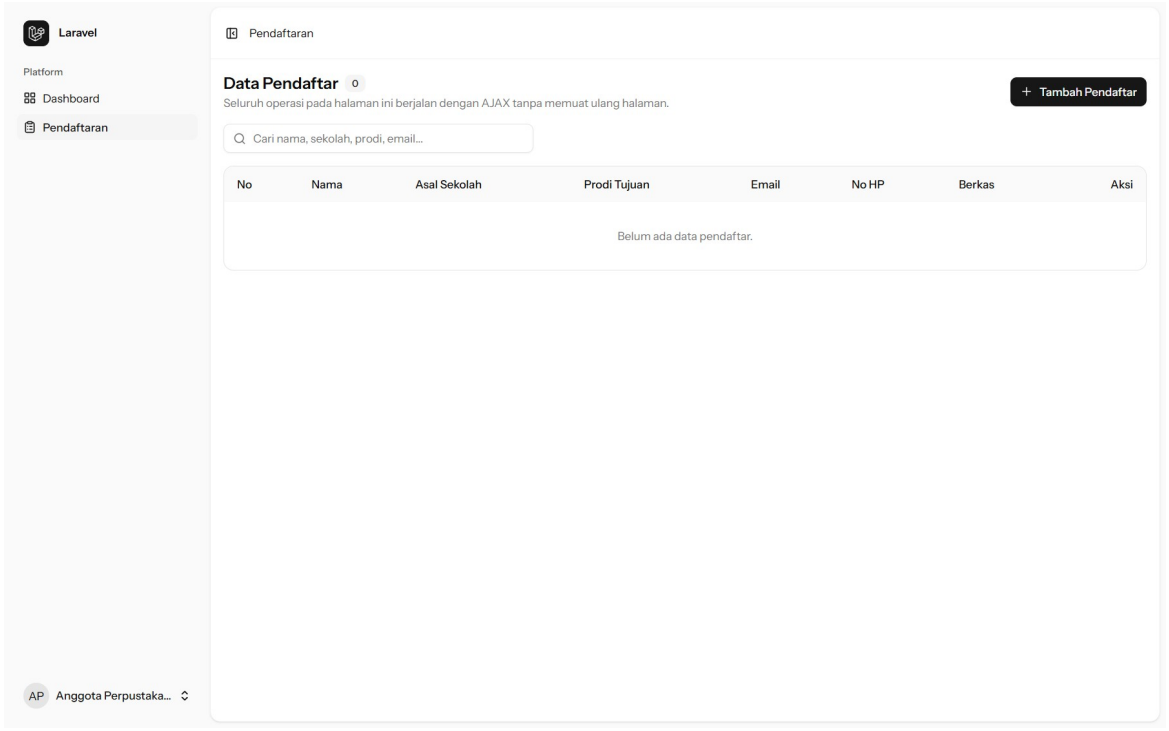
Gambar 5. Dashboard user tanpa kartu jumlah user dan tanpa menu Kelola User

Pembuktian authorization dilakukan dengan membuka /admin/users memakai akun user. Middleware role menghentikan request dan menampilkan halaman 403 dengan pesan yang saya tulis sendiri.

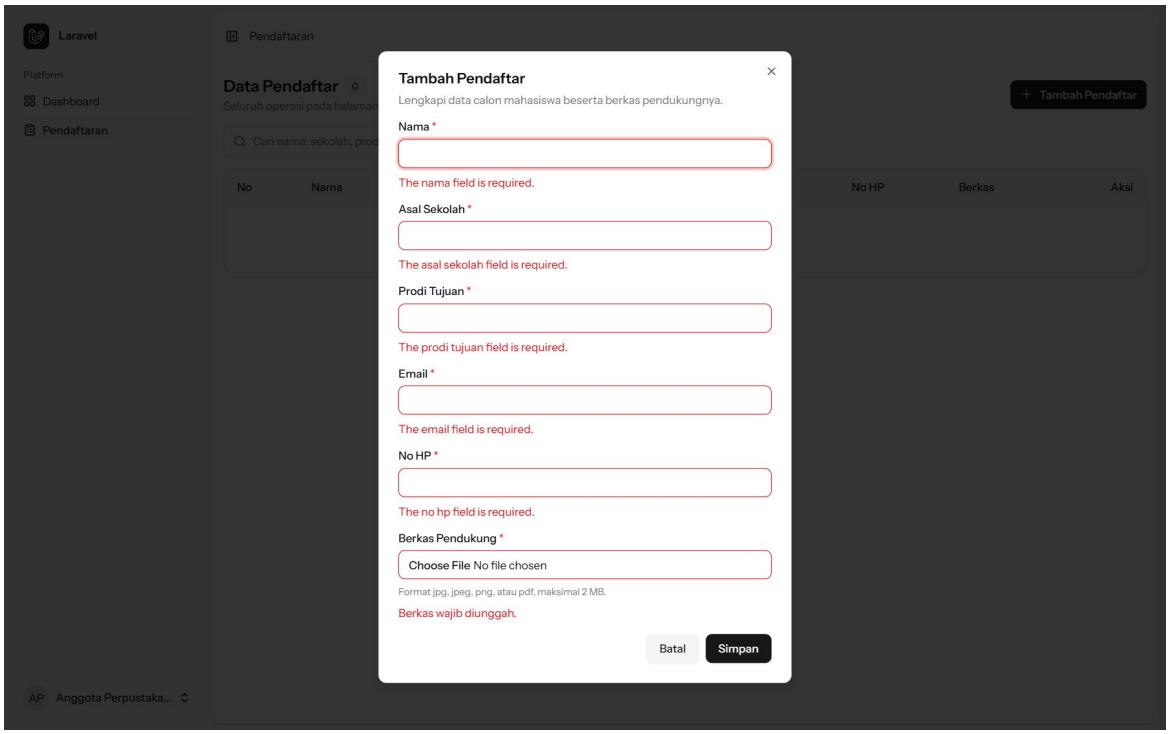


Gambar 6. Error 403 ketika user membuka halaman admin

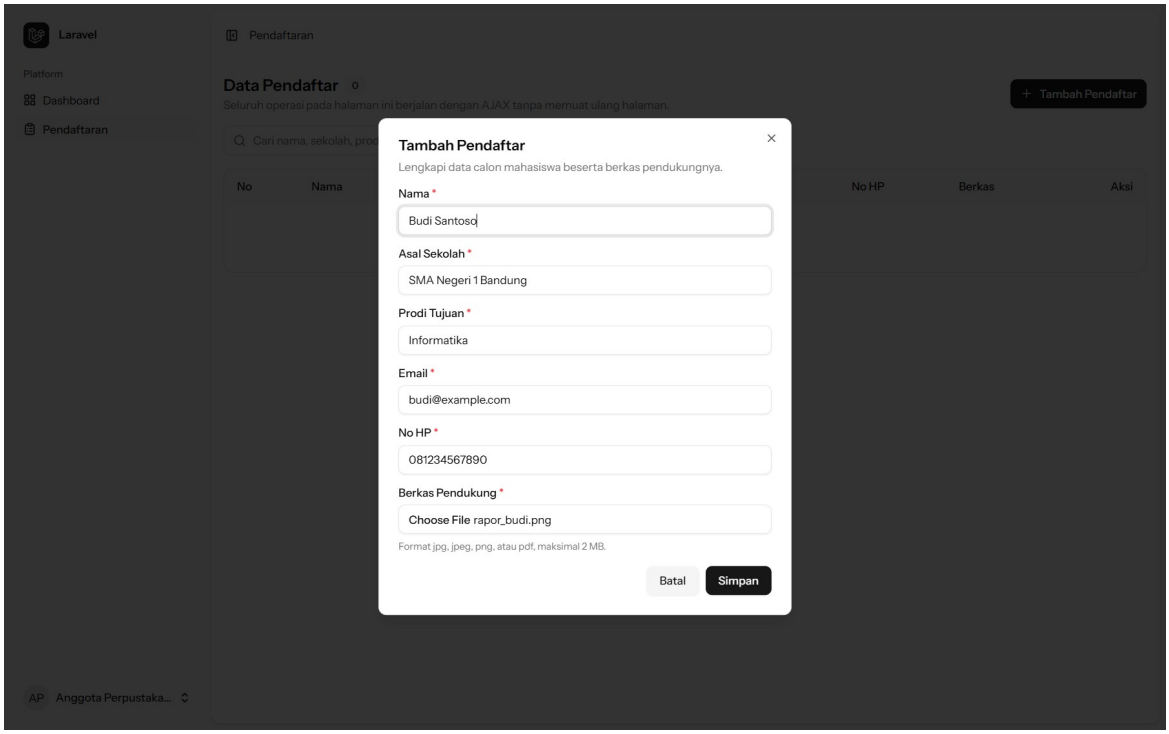
Modul Pendaftaran diuji tanpa memuat ulang halaman. Dialog tambah memakai atribut noValidate supaya validasi dijalankan server dan pesan 422 dapat ditampilkan di bawah input.



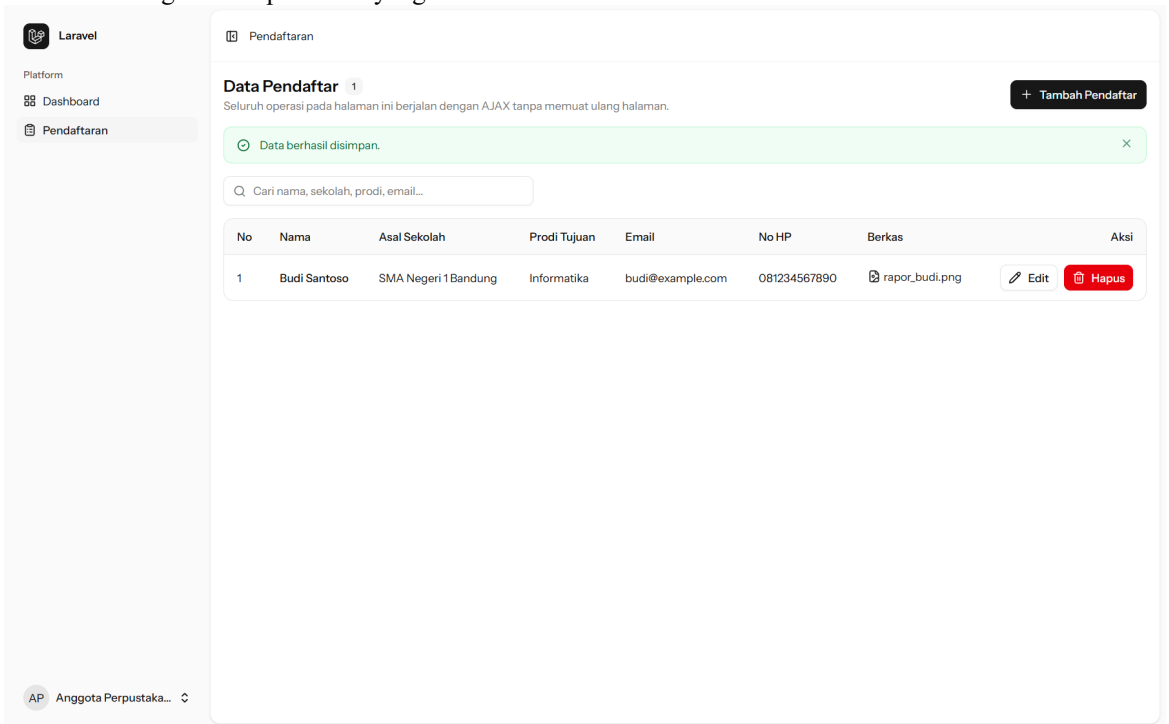
Gambar 7. Halaman Pendaftaran sebelum ada data



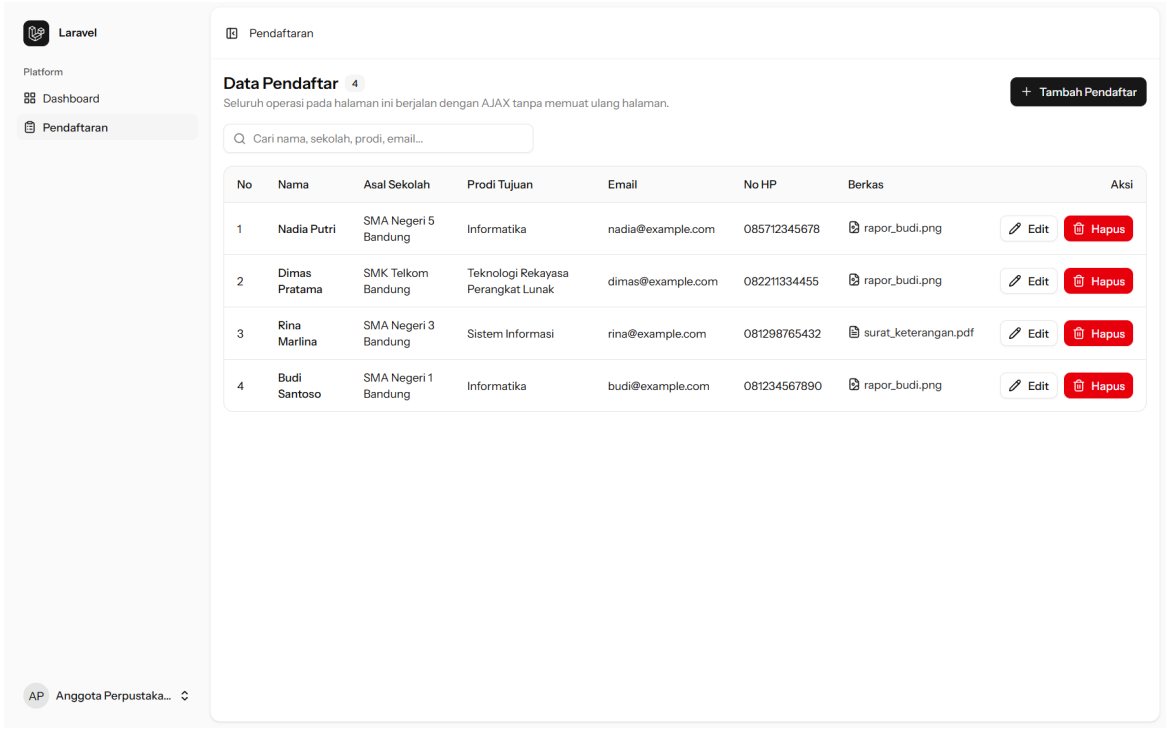
Gambar 8. Pesan validasi 422 di bawah setiap input pada dialog



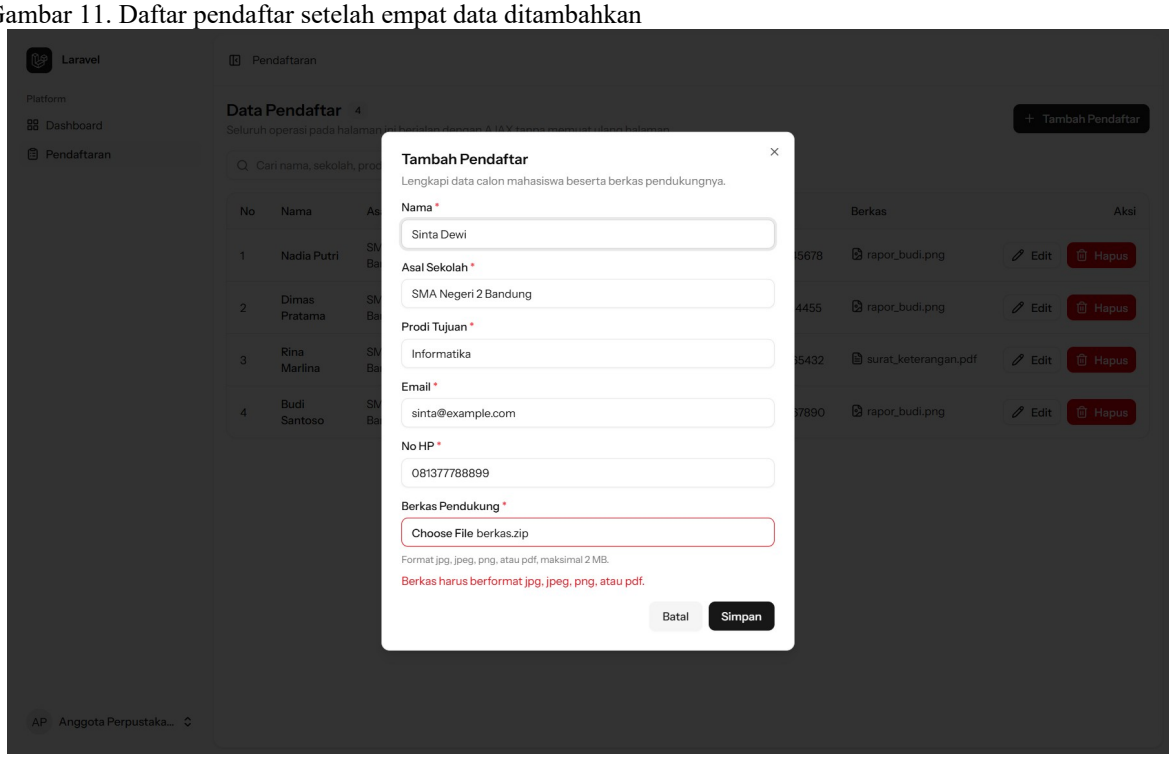
Gambar 9. Dialog tambah pendaftar yang sudah diisi beserta berkas



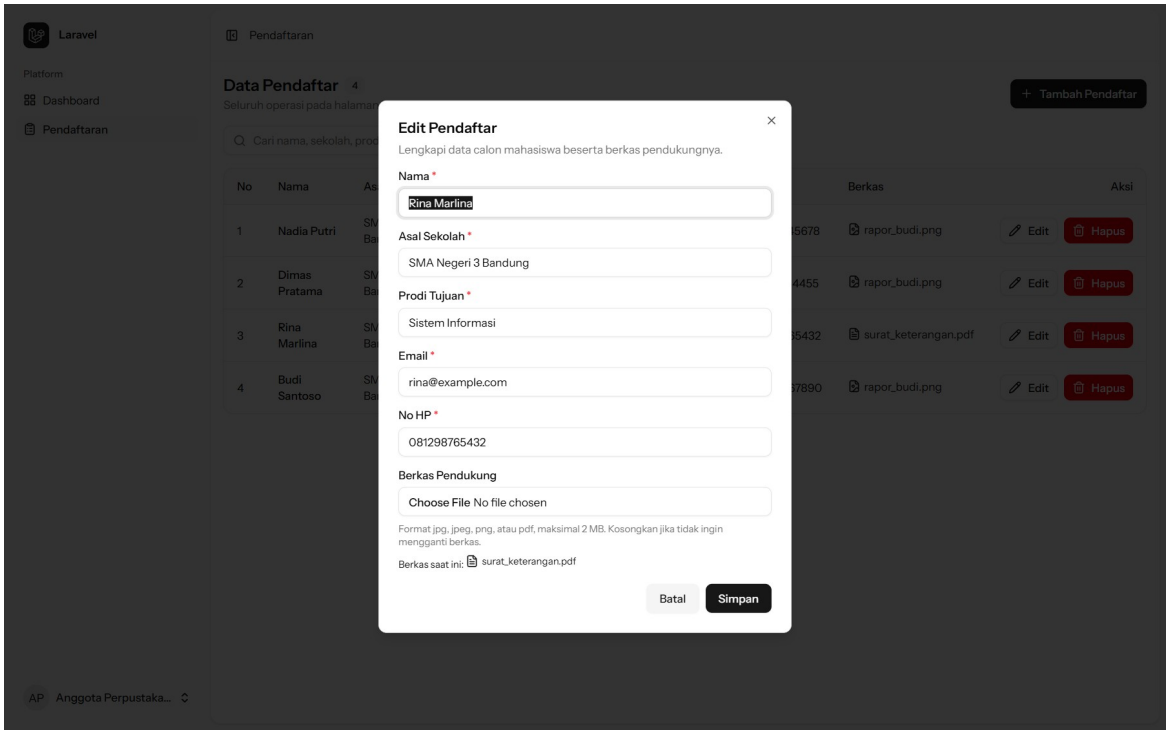
Gambar 10. Pesan berhasil setelah data disimpan tanpa memuat ulang halaman



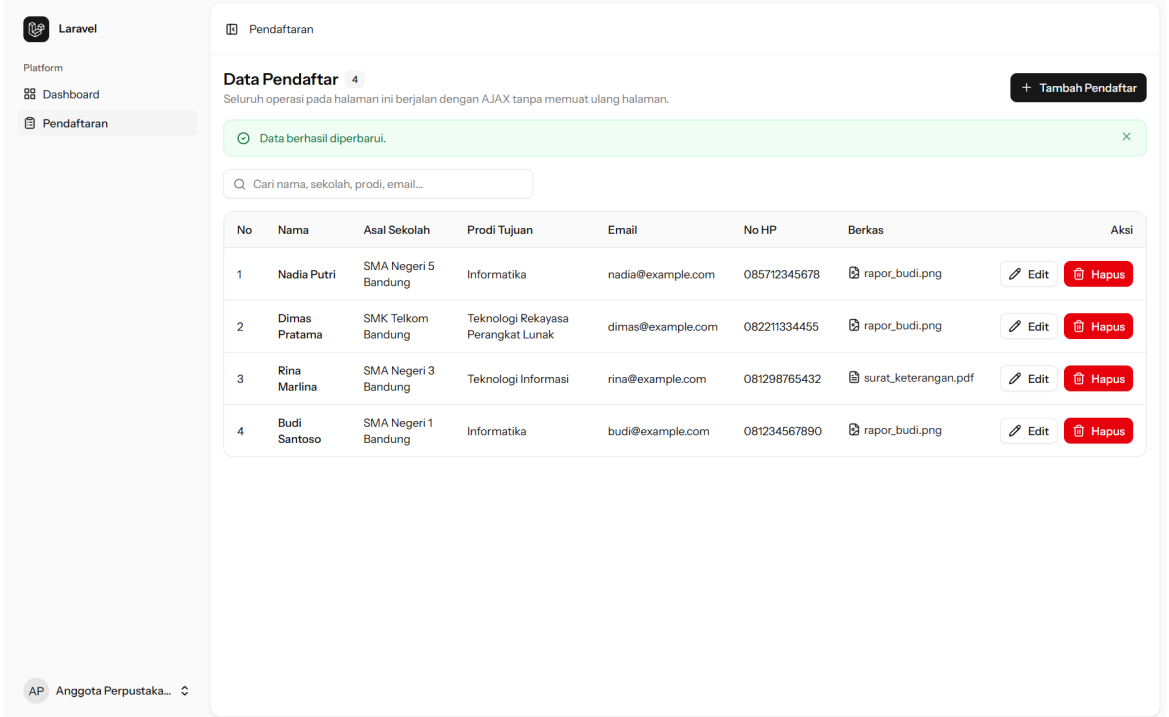
Gambar 11. Daftar pendaftar setelah empat data ditambahkan



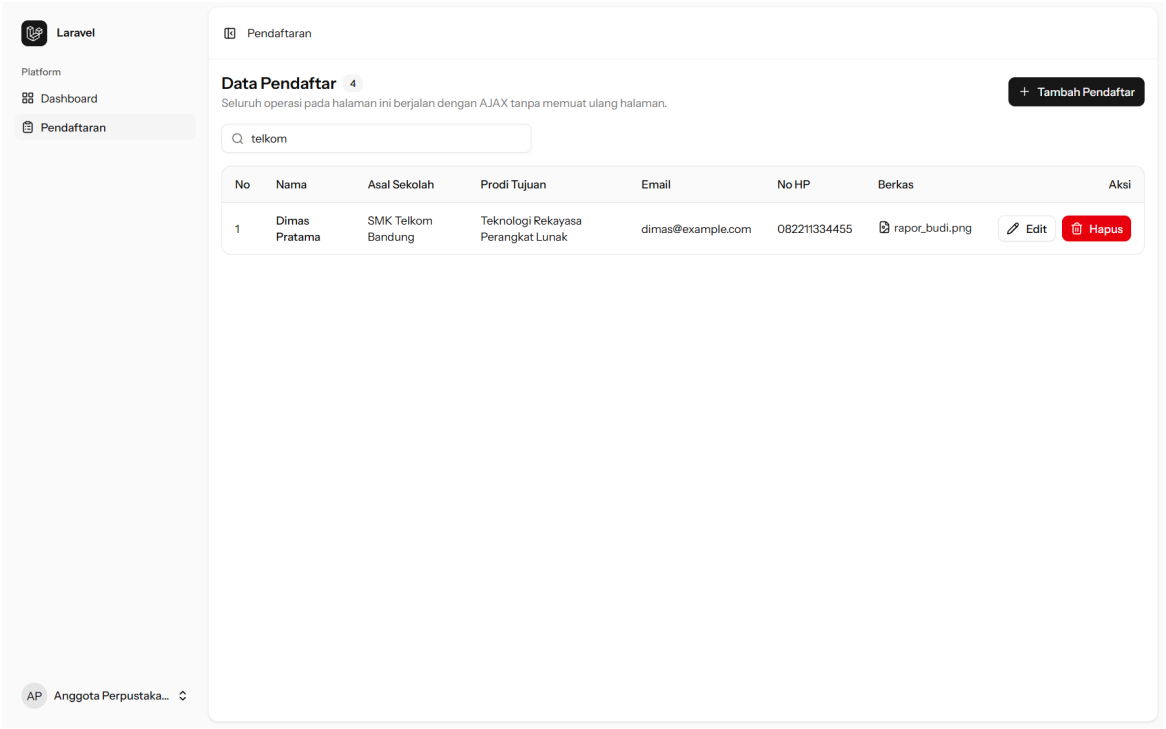
Gambar 12. Berkas ZIP ditolak validasi mimes



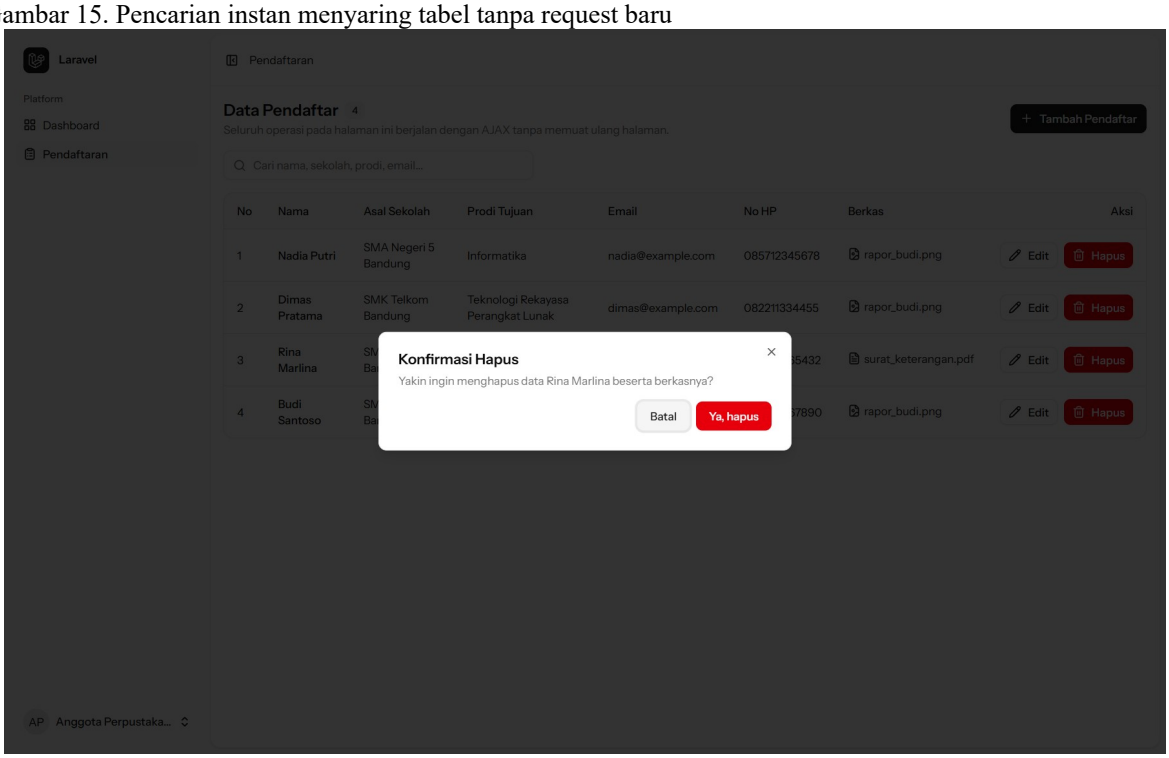
Gambar 13. Dialog edit dengan tautan berkas saat ini



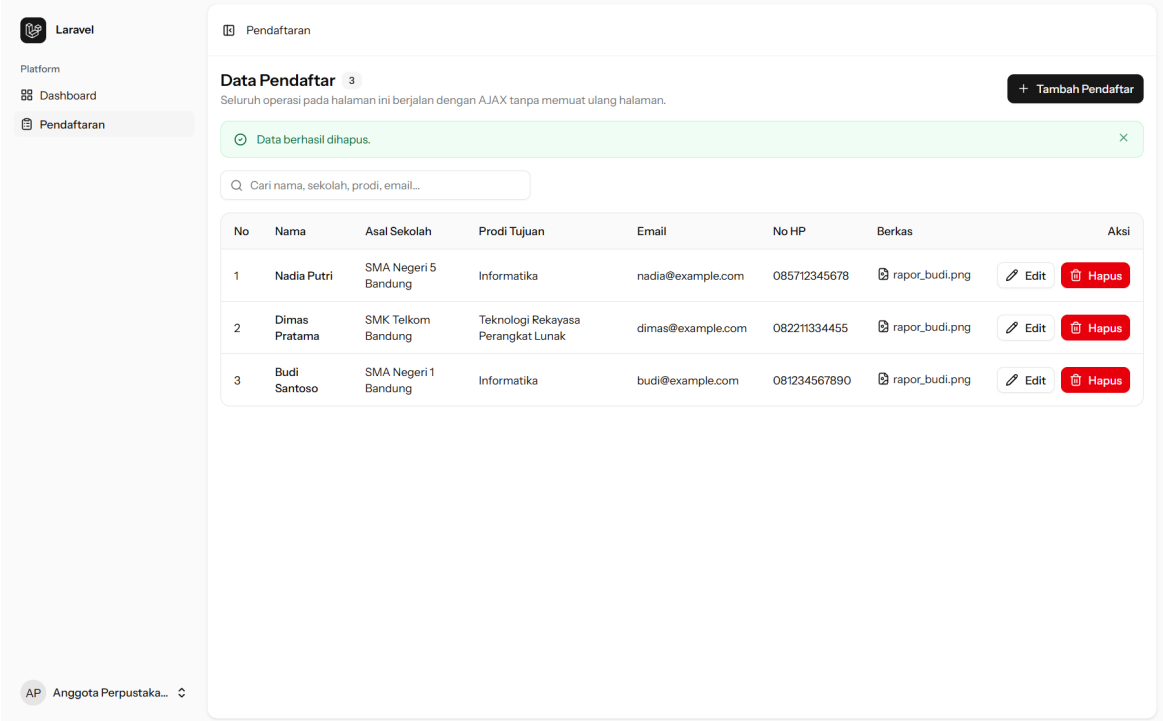
Gambar 14. Pesan berhasil setelah prodi tujuan diubah



Gambar 15. Pencarian instan menyaring tabel tanpa request baru

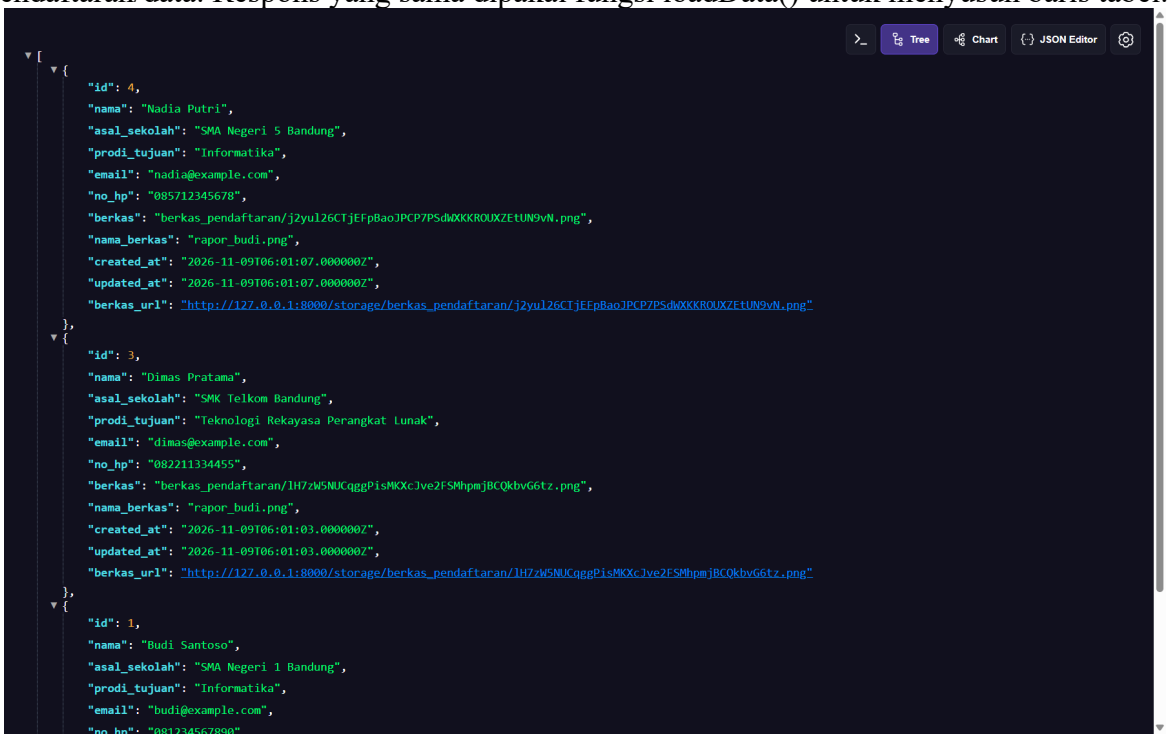


Gambar 16. Dialog konfirmasi sebelum data dihapus



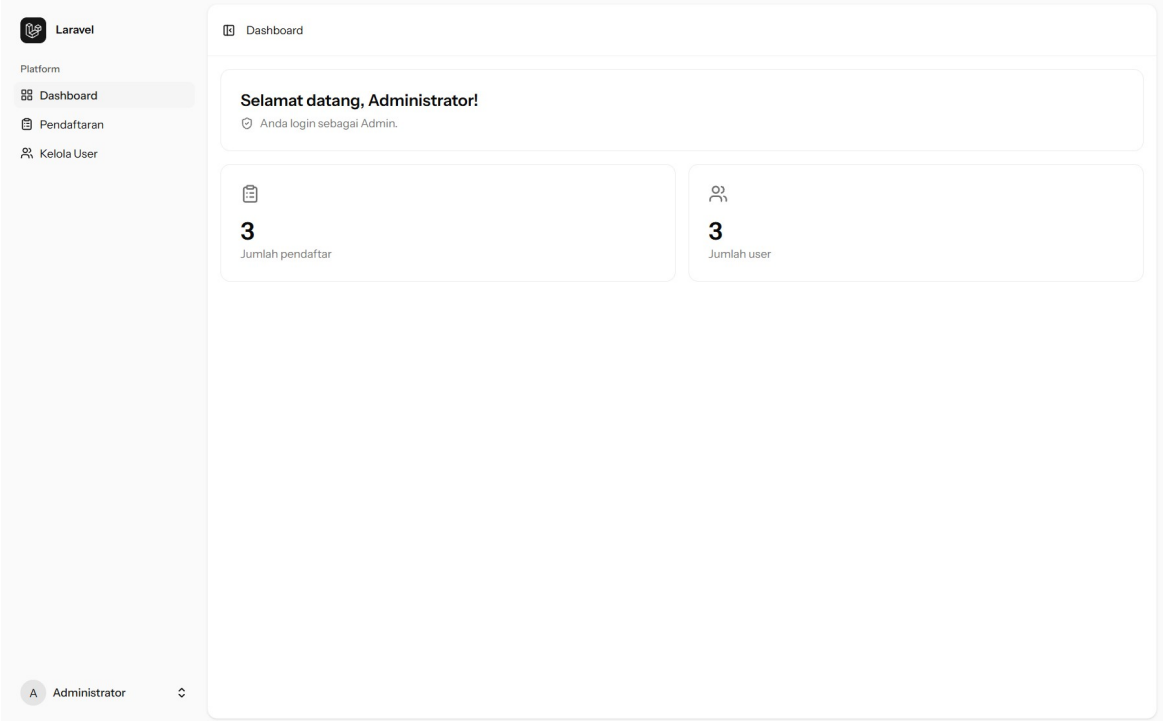
Gambar 17. Pesan berhasil setelah satu data dihapus

Bukti bahwa modul Pendaftaran memakai AJAX saya ambil dari respons JSON endpoint pendaftaran/data. Respons yang sama dipakai fungsi loadData() untuk menyusun baris tabel.



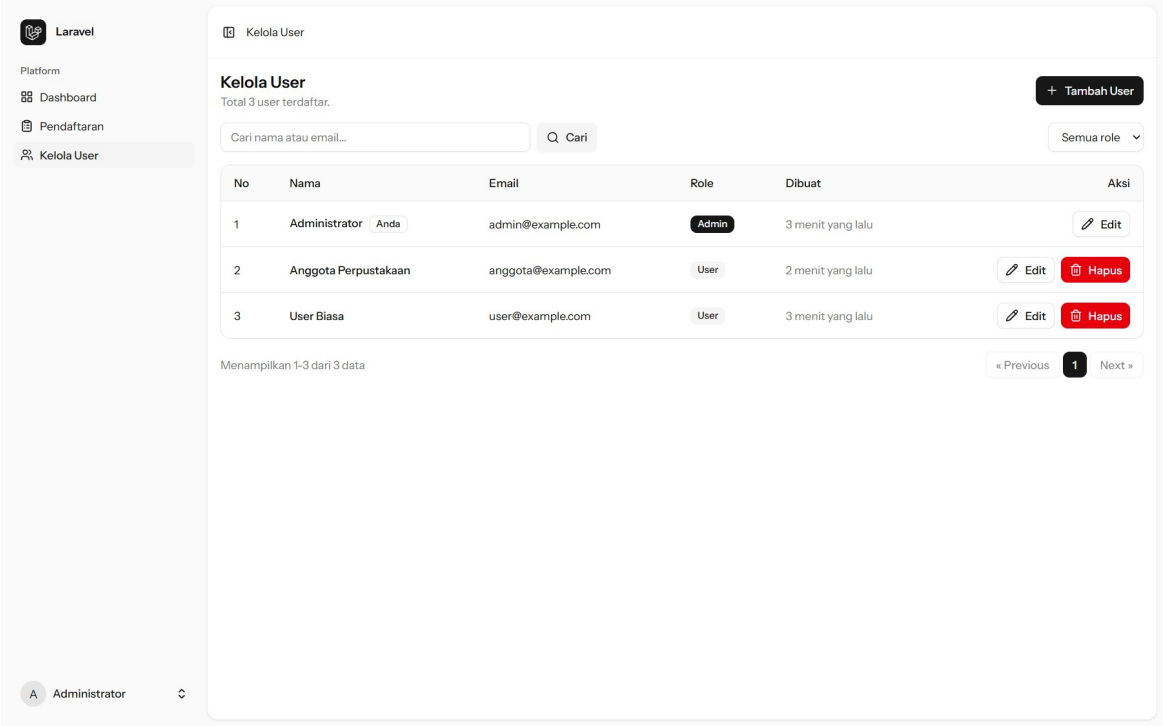
Gambar 18. Respons JSON dari route pendaftaran.data

Setelah login sebagai admin, dashboard menampilkan dua kartu dan sidebar menampilkan menu Kelola User.

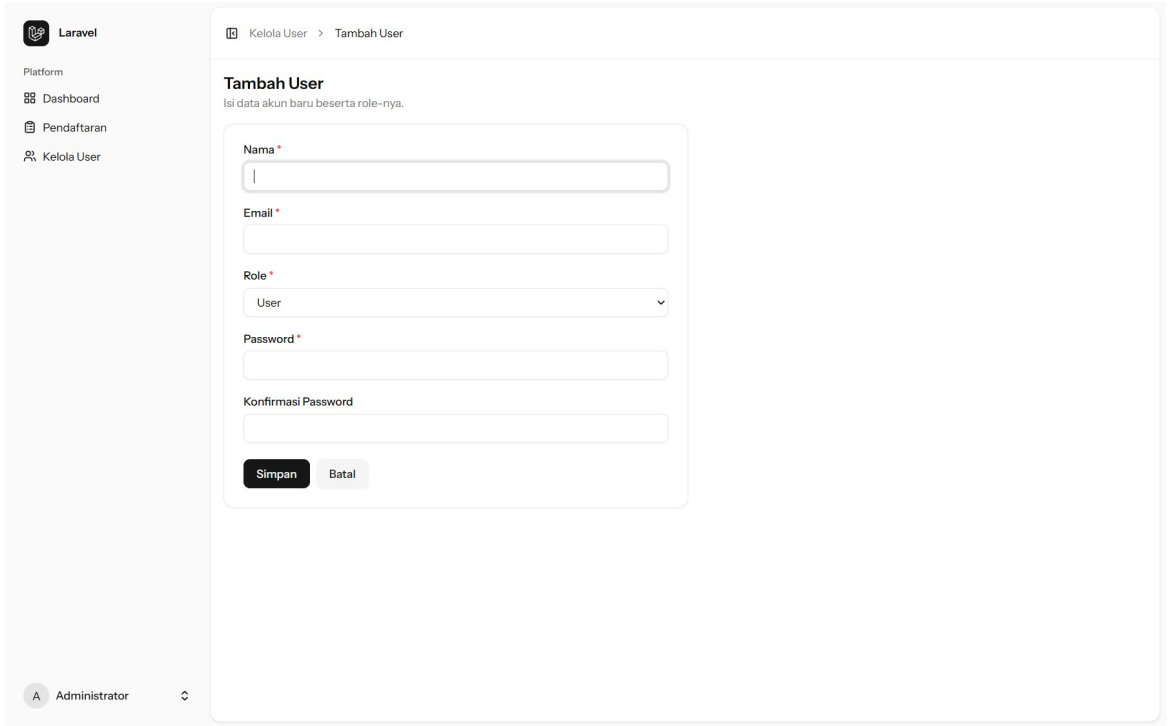


Gambar 19. Dashboard admin dengan kartu jumlah user

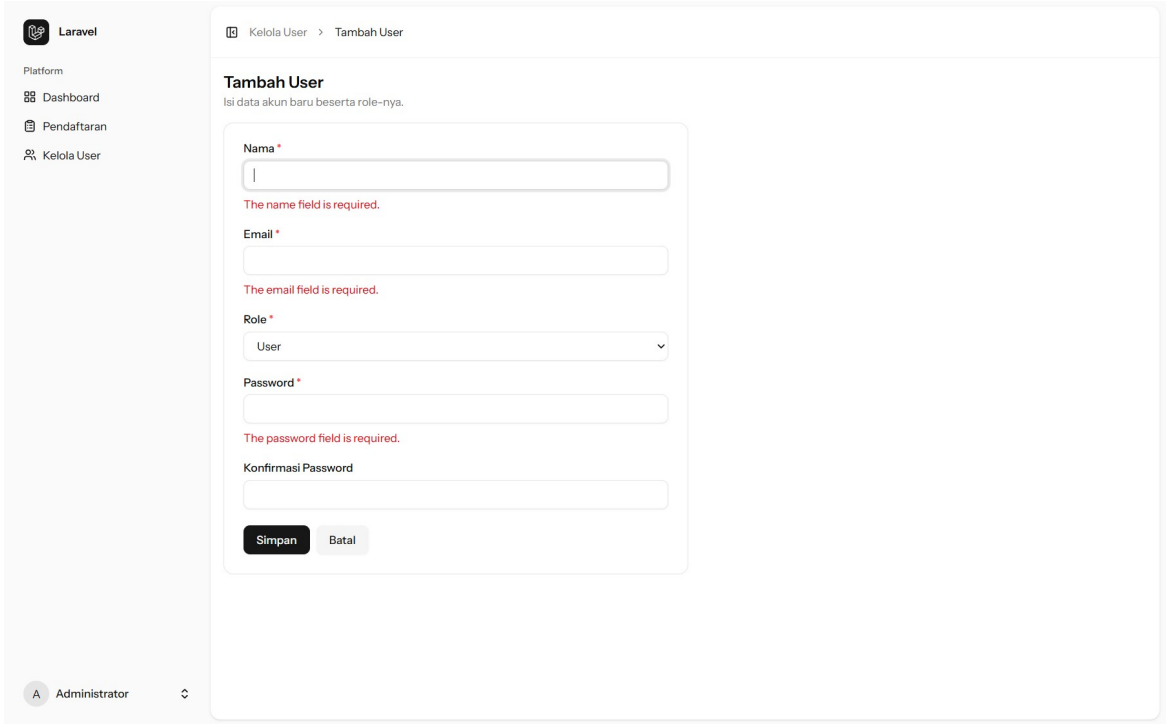
Modul UserManagement menampilkan daftar akun beserta badge role. Akun yang sedang login diberi badge Anda dan tidak memiliki tombol Hapus.



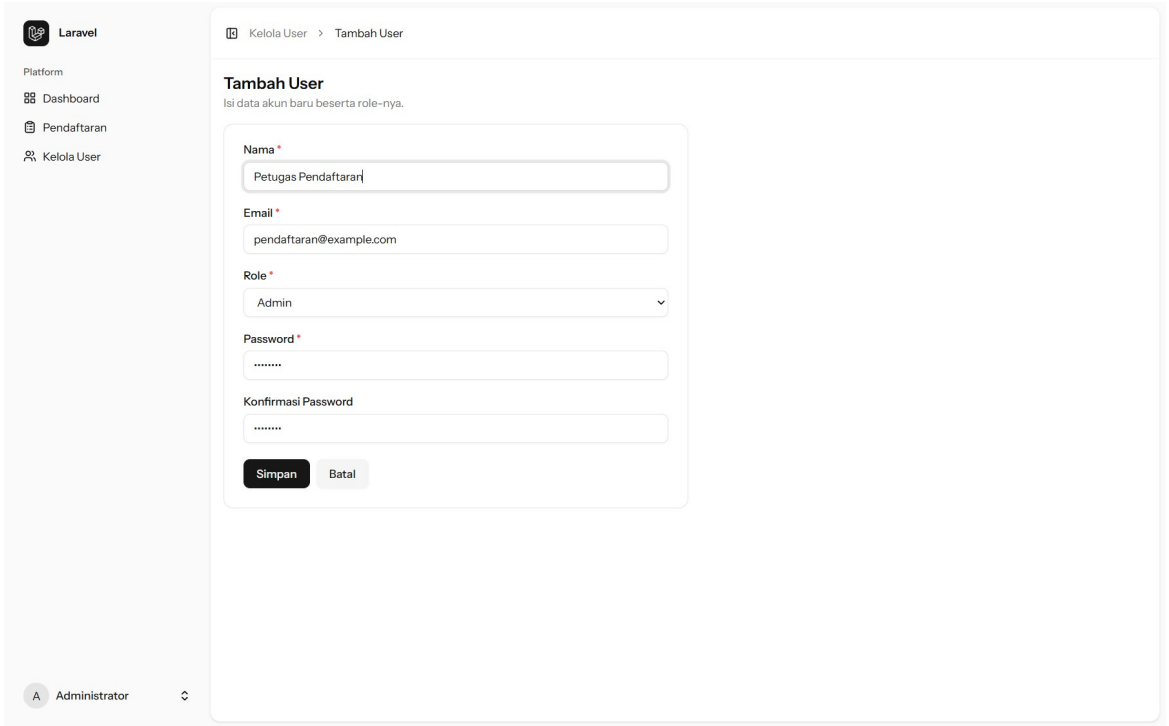
Gambar 20. Daftar user beserta badge role



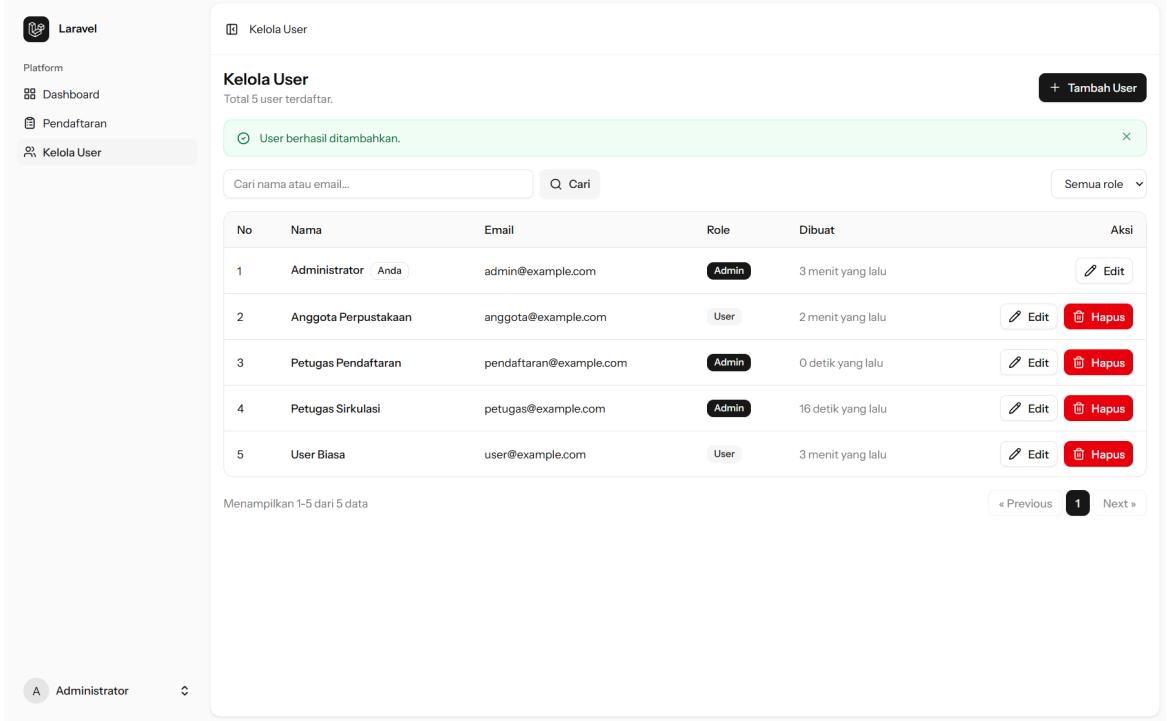
Gambar 21. Form tambah user



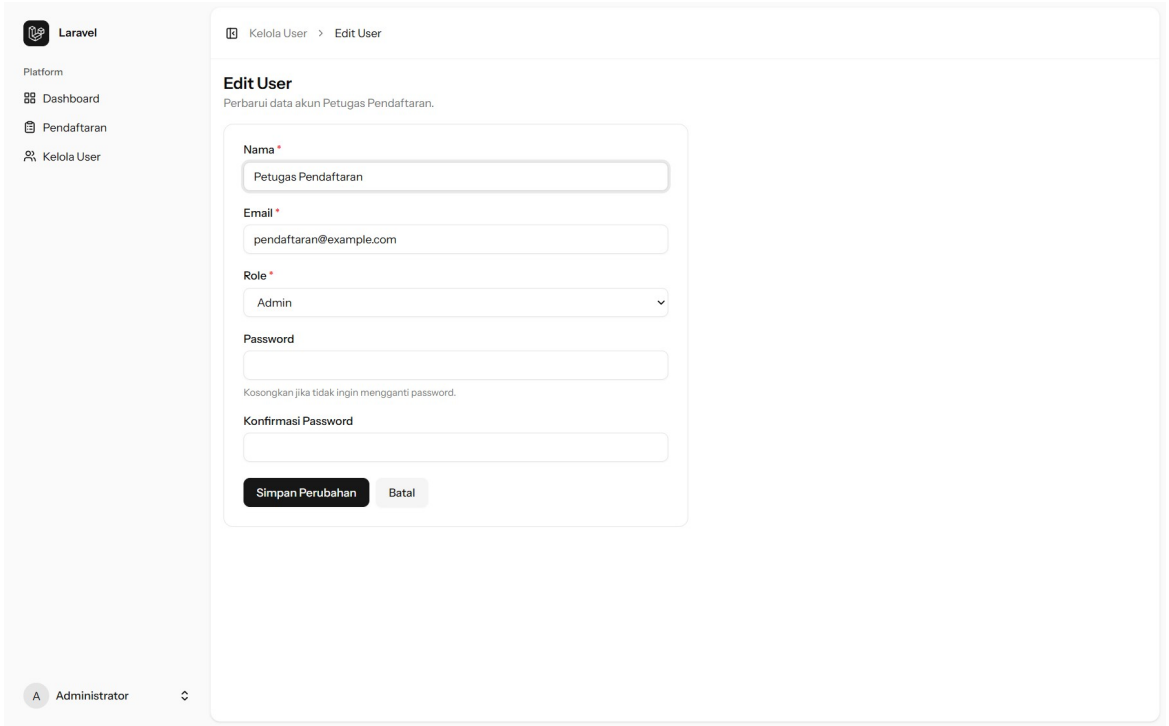
Gambar 22. Pesan validasi pada form tambah user



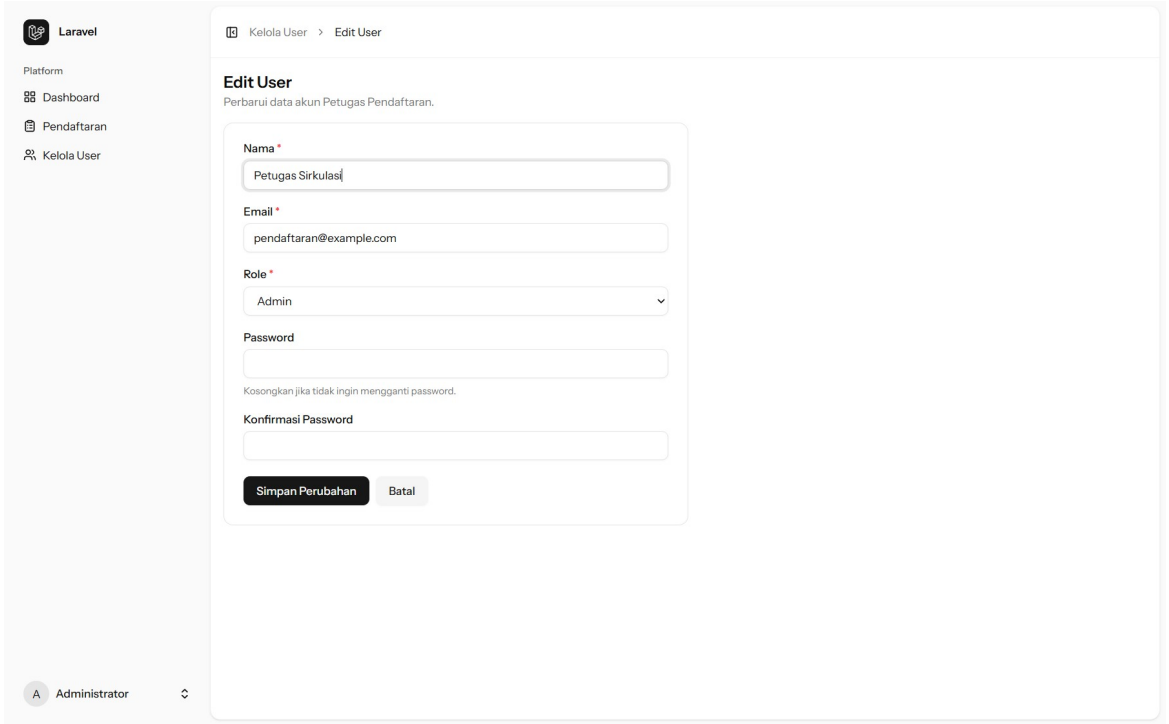
Gambar 23. Form tambah user yang sudah diisi



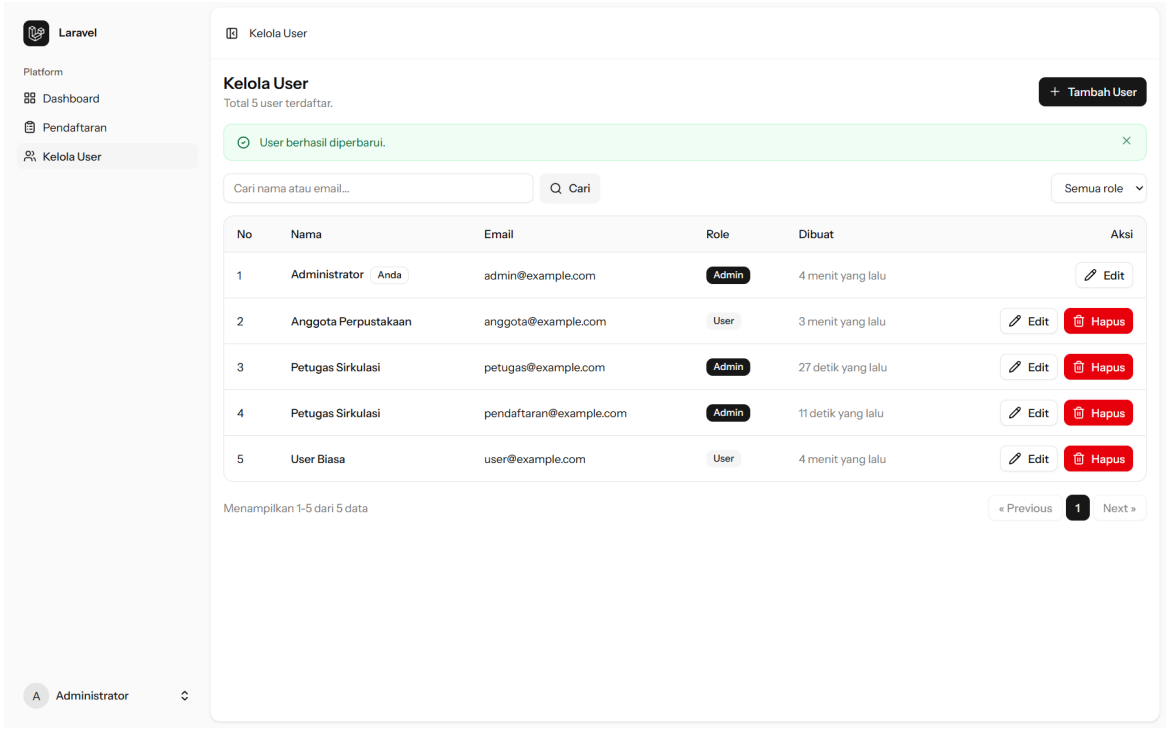
Gambar 24. Pesan berhasil setelah user ditambahkan



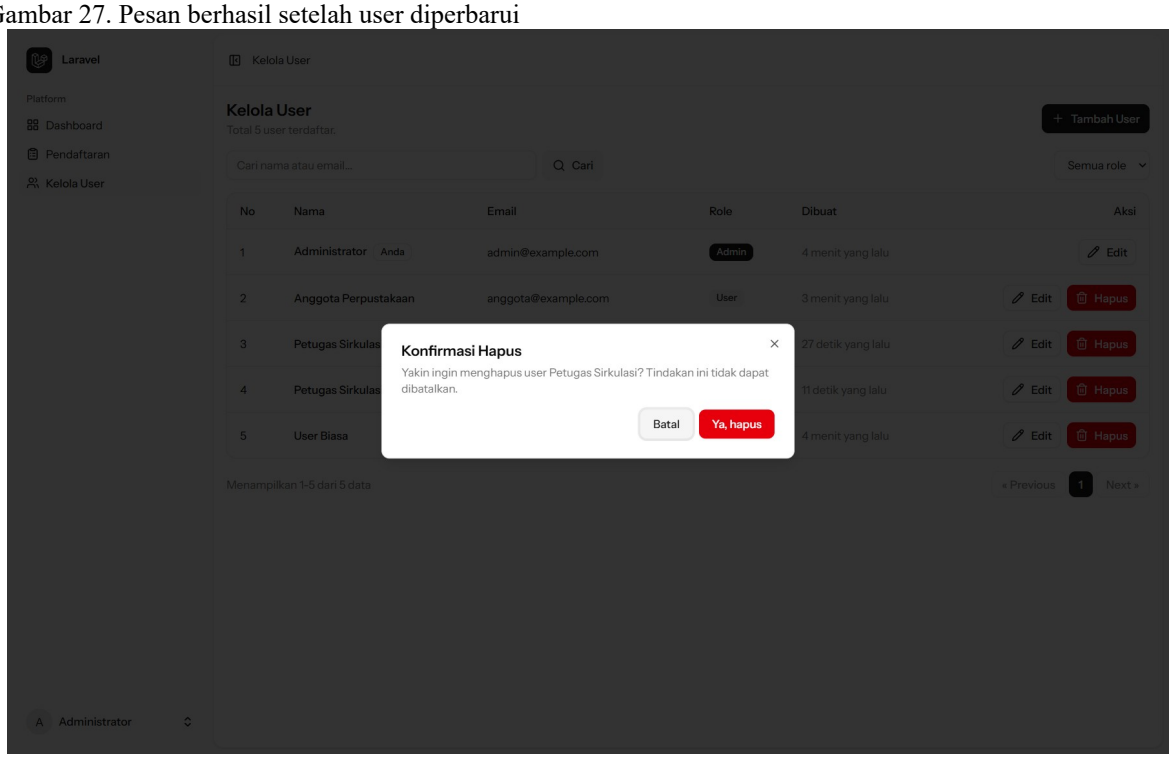
Gambar 25. Form edit user



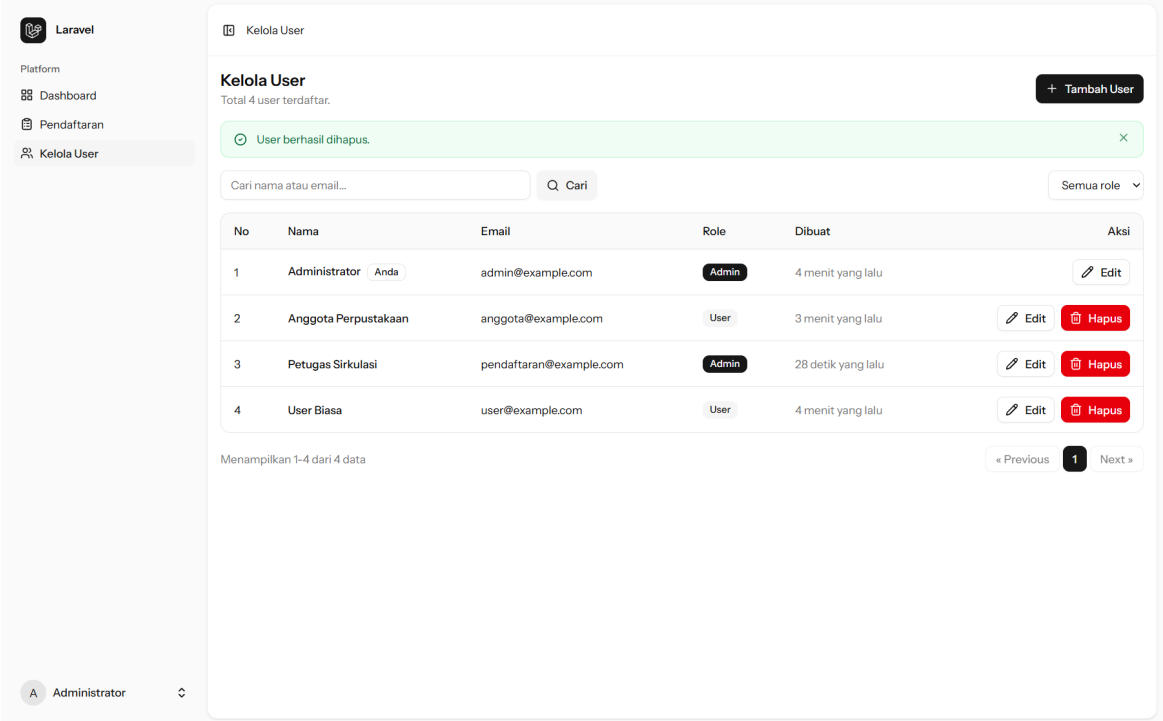
Gambar 26. Nama user diubah dan password dikosongkan



Gambar 27. Pesan berhasil setelah user diperbarui



Gambar 28. Dialog konfirmasi hapus user



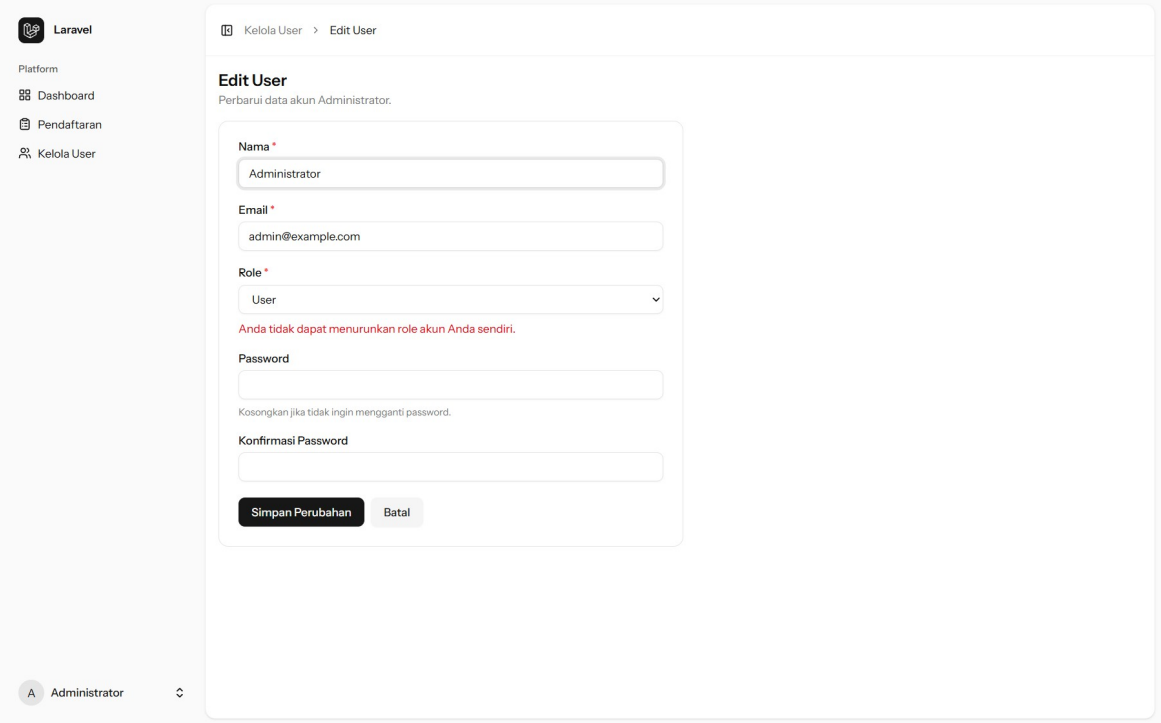
Gambar 29. Pesan berhasil setelah user dihapus

Kolom berkas pada tabel daftars hanya berisi path hasil store(), sedangkan nama asli file disimpan pada kolom nama_berkas. Berkas aslinya berada di storage/app/public/berkas_pendaftaran.

	id	nama	asal_sekolah	prodi_tujuan	email	no_hp	berkas
Edit Copy Delete	1	Budi Santoso	SMA Negeri 1 Bandung	Informatika	budi@example.com	081234567890	berkas_pendaftaran/vRAw8kfcqsEaCftEKB
Edit Copy Delete	3	Dimas Pratama	SMK Telkom Bandung	Teknologi Rekayasa Perangkat Lunak	dimas@example.com	082211334455	berkas_pendaftaran/lH7zW5NUCqggPisMK
Edit Copy Delete	4	Nadia Putri	SMA Negeri 5 Bandung	Informatika	nadia@example.com	085712345678	berkas_pendaftaran/j2yul26CTjEFpBaoJPC

Gambar 30. Tabel daftars pada phpMyAdmin beserta kolom berkas

Aturan Rule::when() pada UpdateUserRequest menolak perubahan role akun sendiri sehingga admin tidak terkunci dari menu admin.



Gambar 31. Admin tidak dapat menurunkan role akunnya sendiri

Selain tangkapan layar halaman, saya menyertakan keluaran perintah yang dipakai untuk memeriksa route, status modul, isi folder berkas, dan hasil pengujian.

```
php artisan route:list --except-vendor

GET|HEAD .well-known/passkey-endpoints .. well-known.passkeys › routes/settings.php:29
GET|HEAD / .. home › routes/web.php:8
GET|HEAD admin/users admin.users.index ›
   Modules\UserManagement\Http\Controllers\UserManagementController@in…
POST admin/users admin.users.store ›
   Modules\UserManagement\Http\Controllers\UserManagementController@st…
GET|HEAD admin/users/create admin.users.create ›
   Modules\UserManagement\Http\Controllers\UserManagementContr…
PUT|PATCH admin/users/{user} admin.users.update ›
   Modules\UserManagement\Http\Controllers\UserManagementContr…
DELETE admin/users/{user} admin.users.destroy ›
   Modules\UserManagement\Http\Controllers\UserManagementCont…
GET|HEAD admin/users/{user}/edit admin.users.edit ›
   Modules\UserManagement\Http\Controllers\UserManagementCo…
GET|HEAD dashboard .. dashboard › DashboardController
GET|HEAD pendaftaran .. pendaftaran.index ›
   Modules\Pendaftaran\Http\Controllers\PendaftaranController@index
POST pendaftaran .. pendaftaran.store ›
   Modules\Pendaftaran\Http\Controllers\PendaftaranController@store
GET|HEAD pendaftaran/data pendaftaran.data ›
   Modules\Pendaftaran\Http\Controllers\PendaftaranController@getD…
PUT pendaftaran/{daftar} pendaftaran.update ›
   Modules\Pendaftaran\Http\Controllers\PendaftaranControlle…
DELETE pendaftaran/{daftar} pendaftaran.destroy ›
   Modules\Pendaftaran\Http\Controllers\PendaftaranControll…
GET|HEAD pendaftaran/{daftar}/edit pendaftaran.edit ›
   Modules\Pendaftaran\Http\Controllers\PendaftaranContro…
ANY settings .. Illuminate\Routing › RedirectController
PUT settings/password .. user-password.update › Settings\SecurityController@update
GET|HEAD settings/profile .. profile.edit › Settings\ProfileController@edit
PATCH settings/profile .. profile.update › Settings\ProfileController@update
DELETE settings/profile .. profile.destroy › Settings\ProfileController@destroy
GET|HEAD settings/security .. security.edit › Settings\SecurityController@edit

Showing [21] routes
```

Gambar 32. Hasil php artisan route:list --except-vendor

```
php artisan module:list

Status / Name .. Path / priority
[Enabled] Pendaftaran .. D:\laragon\www\PABW\pabw-9\pertemuan9_hmvc_react_2\Modules/Pendaftaran
    [0]
[Enabled] UserManagement ..
   D:\laragon\www\PABW\pabw-9\pertemuan9_hmvc_react_2\Modules/UserManagement [0]
```

Gambar 33. Hasil php artisan module:list

```
dir storage\app\public\berkas_pendaftaran

total 22110
-rwxrwxrwx 1 somebody somegroup 7370 Nov  9 13:01 j2yul26CTjEFpBaoJPCP7PSdWXKKROUXZEtUN9vN.png
-rwxrwxrwx 1 somebody somegroup 7370 Nov  9 13:01 lH7zW5NUCqggPisMKXcJve2FSMhpmjBCQkbvG6tz.png
-rwxrwxrwx 1 somebody somegroup 7370 Nov  9 13:00 vRAw8kfcqsEaCftEKBJtaSxYBzU1G6pTcUA7KOv7.png
```

Gambar 34. Isi folder storage/app/public/berkas_pendaftaran

```
php vendor/pestphp/pest/bin/pest --testdox

  PASS  Tests\Feature\PendaftaranTest
✓ index renders the react page of the module
✓ data endpoint returns registrations as json
✓ store returns 422 when the form is empty
✓ store saves the data and the uploaded file
✓ zip file is rejected
✓ update replaces the old file
✓ destroy deletes the data and its file, unknown id returns 404
  PASS  Tests\Feature\UserManagementTest
✓ guest is redirected to login from module pages
✓ registered user always gets the user role
✓ user cannot open the user management page
✓ admin can see the user list rendered by the module page
✓ admin can create a user
✓ admin cannot demote or delete their own account
  Tests:    53 passed (199 assertions)
  Duration: 3.50s
OK (53 tests, 199 assertions)
```

Gambar 35. Hasil php artisan test

5. Analisis dan Pembahasan

Pemisahan kode ke dalam modul membuat controller, request, model, migration, route, dan halaman React milik satu fitur berada di satu folder. Modul UserManagement dan Pendaftaran tidak perlu didaftarkan pada routes/web.php karena masing-masing memuat route-nya melalui service provider. Pada sisi tampilan, Inertia menjadi penghubung antara route Laravel dan halaman React: controller memanggil Inertia::render dengan nama NamaModul::halaman, lalu import.meta.glob pada app.tsx memetakan nama itu ke file .tsx di dalam folder modul. Tanpa pengaturan resolve tersebut, halaman modul tidak ditemukan walaupun file-nya ada.

Authentication dan authorization dipisahkan dengan jelas. Fortify memeriksa email dan password, sedangkan hak akses ditentukan dari kolom role yang di-cast ke enum UserRole. Middleware role bekerja pada level route, sehingga user tidak dapat membuka halaman admin walaupun alamatnya diketik langsung. Menu Kelola User yang disembunyikan pada sidebar hanya pengaturan tampilan, bukan perlindungan utama. Perlindungan sebenarnya ada pada middleware dan pada method authorize() di Form Request.

Perbedaan CRUD Inertia dan CRUD AJAX terlihat pada siapa yang mengatur data halaman. Pada modul UserManagement, controller mengembalikan Inertia::render() atau redirect beserta pesan flash, dan Inertia mengirim ulang props setelah setiap aksi. Pada modul Pendaftaran, controller mengembalikan JSON, lalu state React diperbarui dari respons itu.

Konsekuensinya, penanganan error juga berbeda: form Inertia membaca errors dari useForm(), sedangkan form AJAX membaca xhr.responseJSON.errors atau error.data.errors ketika status respons 422.
Pengiriman berkas pada AJAX memerlukan FormData karena JSON tidak dapat membawa file. Helper http.ts sengaja tidak menulis header Content-Type agar browser mengisi multipart/form-data beserta boundary-nya, setara dengan processData: false dan contentType: false pada jQuery di Modul 8. Di sisi server, berkas disimpan dengan store('berkas_pendaftaran', 'public') sehingga nama file dibuat acak, sedangkan database hanya menyimpan path dan nama aslinya. Pola ini membuat nama file tidak dapat ditebak, tetapi berkas tetap dapat dibuka melalui symbolic link public/storage.
Beberapa kendala benar-benar muncul selama pengerjaan dan sudah saya perbaiki. Pertama, muncul Class "Modules\Pendaftaran\Providers\PendaftaranServiceProvider" not found berulang kali pada storage/logs/laravel.log. Penyebabnya, bagian merge-plugin belum ada di composer.json sehingga composer.json milik modul belum ikut dibaca. Setelah bagian itu ditambahkan, composer dump-autoload dijalankan dan modul dapat dimuat.
Kedua, halaman modul berhenti dengan error 500 Unable to locate file in Vite manifest: resources/js/pages/Pendaftaran::index.tsx. Penyebabnya ada pada resources/views/app.blade.php yang menambahkan halaman aktif sebagai entri Vite, padahal halaman modul tidak punya entri di manifest karena dimuat lewat import.meta.glob. Saya mengubah file itu agar entri per halaman hanya dipakai untuk halaman di luar modul. Error yang sama juga membuat satu pengujian PendaftaranTest gagal dengan status 500 sebelum diperbaiki.
Ketiga, satu pengujian sempat tidak pernah dijalankan. File pengujian role dan UserManagement awalnya saya simpan sebagai tests/Feature/UserManagement.php, sehingga hanya 47 pengujian yang dieksekusi. Pest hanya memuat file yang namanya berakhiran Test.php, jadi file itu saya ganti nama menjadi UserManagementTest.php dan jumlah pengujian menjadi 53. Kejadian ini menunjukkan bahwa jumlah pengujian yang lulus perlu dicocokkan dengan jumlah test() yang ditulis, karena file yang salah nama tidak memunculkan error apa pun.
Keempat, MySQL sempat berhenti di tengah pengujian browser sehingga halaman menampilkan SQLSTATE[HY000] [2002] Connection refused. Setelah layanan MySQL dinyalakan kembali, aplikasi berjalan normal tanpa perubahan kode. Selain itu ada dua hal yang berbeda dari teks modul dan perlu saya sebutkan. Pesan validasi bawaan Laravel untuk atribut selain berkas dan no_hp masih tampil dalam bahasa Inggris, misalnya The nama field is required, karena terjemahan atribut tidak dibuat. Pada PendaftaranTest saya juga menambahkan helper fakePublicDisk() agar assertExists() dan assertMissing() tetap terbaca oleh PHPStan, karena Storage::disk() bertipe kontrak Filesystem.

6. Kesimpulan

Aplikasi HMVC React berhasil dijalankan dengan dua modul. Modul UserManagement menangani CRUD berbasis Inertia khusus admin, sedangkan modul Pendaftaran menangani CRUD AJAX dengan unggah berkas yang dapat dibuka kembali, diganti saat edit, dan ikut terhapus bersama datanya. Halaman React kedua modul berada di dalam folder modul dan dipanggil dengan format NamaModul::halaman.
Authentication memakai Fortify, sedangkan pembatasan hak akses memakai middleware role yang mendukung satu atau beberapa role sekaligus. Middleware itu berhasil menolak user yang membuka halaman admin dengan kode 403, dan aturan pada UpdateUserRequest serta destroy() mencegah admin menghapus atau menurunkan role akunnya sendiri.
Pengujian otomatis dengan Pest berjalan pada database kosong dan mencakup role, middleware, CRUD Inertia, CRUD AJAX, validasi 422, penolakan format berkas,

penggantian berkas, serta error 404. Dari pengerjaan ini saya memahami bahwa halaman modul memerlukan pengaturan resolve pada app.tsx dan pengecualian pada entri Vite, serta bahwa nama file pengujian menentukan apakah pengujian itu dijalankan atau tidak.

7. Link Repository Github
https://github.com/dhodhoo/pabw-andhika-yuridho